

Learning Specialized Activation Functions for Physics-Informed Neural Networks

Honghui Wang¹, Lu Lu², Shiji Song¹ and Gao Huang^{1,*}

¹ Department of Automation, Tsinghua University, Beijing 100084, P.R. China.

² Department of Statistics and Data Science, Yale University, New Haven, CT 06511, USA.

Received 28 February 2023; Accepted (in revised version) 9 August 2023

Abstract. Physics-informed neural networks (PINNs) are known to suffer from optimization difficulty. In this work, we reveal the connection between the optimization difficulty of PINNs and activation functions. Specifically, we show that PINNs exhibit high sensitivity to activation functions when solving PDEs with distinct properties. Existing works usually choose activation functions by inefficient trial-and-error. To avoid the inefficient manual selection and to alleviate the optimization difficulty of PINNs, we introduce adaptive activation functions to search for the optimal function when solving different problems. We compare different adaptive activation functions and discuss their limitations in the context of PINNs. Furthermore, we propose to tailor the idea of learning combinations of candidate activation functions to the PINNs optimization, which has a higher requirement for the smoothness and diversity on learned functions. This is achieved by removing activation functions which cannot provide higher-order derivatives from the candidate set and incorporating elementary functions with different properties according to our prior knowledge about the PDE at hand. We further enhance the search space with adaptive slopes. The proposed adaptive activation function can be used to solve different PDE systems in an interpretable way. Its effectiveness is demonstrated on a series of benchmarks. Code is available at <https://github.com/LeapLabTHU/AdaAFforPINNs>.

AMS subject classifications: 65M99, 68T07

Key words: Partial differential equations, deep learning, adaptive activation functions, physics-informed neural networks.

1 Introduction

Recent years have witnessed the remarkable progress of physics-informed neural networks (PINNs) on modeling the dynamics of physics systems [1–6]. The underlying

*Corresponding author. Email addresses: wanghh20@mails.tsinghua.edu.cn (H. Wang), lu.lu@yale.edu (L. Lu), shijis@mail.tsinghua.edu.cn (S. Song), gaohuang@tsinghua.edu.cn (G. Huang)

physics laws, usually presented as ordinary and partial differential equations (ODEs and PDEs), are embedded as soft constraints to guide the learning process of deep neural networks [1, 7, 8]. Despite the effectiveness, the introduction of PDE-based loss function makes the optimization more ill-conditioned [9, 10]. Efforts have been made to alleviate this problem from the aspects of loss weight balancing [9, 11], loss function design [12–14], adaptive collocation point sampling [15–18], domain decomposition [19–21], and curriculum learning [10]. However, since most previous works adopt the standard fully-connected networks, the relationship between network architectures and the optimization difficulty of PINNs is less explored, regardless of the fact that the advancement of network architectures is the key to the success on deep learning in computer vision [22, 23] and natural language processing [24].

At the heart of network architectures lie the activation functions, which play a significant role in the expressiveness and optimization of models. Recent works [25–29] have observed that the choice of activation functions affects the learning of continuous signal representations. For example, the hyperbolic tangent function is shown to suffer from numerical instability when simulating vortex induced vibrations, while a PINN with sinusoidal function can be optimized smoothly [25]. Another important observation is that the optimal activation function depends on the problem at hand. While the Rectified Linear Unit [30–33] is widely adopted in most computer vision and natural language processing tasks [34], there is no such default choice of activation functions for PINNs when applied to physical systems with distinct properties. In fact, PINNs show great sensitivity to activation functions. These observations reveal the possibility and necessity of reducing the training difficulty of PINNs by selecting an appropriate activation function.

The various characteristics of different PDE systems make the choice of activation functions a critical aspect in PINNs. The common practice to find the optimal activation functions is by trial-and-error, which requires extensive computational resources and human knowledge. As an alternative, the adaptive activation functions are designed to find specialized activation functions for different architectures and tasks. In this work, we aim to explore different adaptive activation functions and discuss their limitations in the context of PINNs. The major difference of existing methods lies in the search space. The piece-wise linear function is adopted as the universal function approximator in some methods, such as APL [35], RePLU [36] and PWLU [37]. The formulation of SLAF [38] is based on Taylor approximation with polynomial basis. PAU [39] leverages Padé approximation to form its search space. ACON [40] is proposed as a smooth approximator to the general Maxout family activation functions [41]. Built upon previous works [42–44] to learn activation functions as linear combinations of candidate activation functions, the Adaptive Blending Units (ABU) [45] explore different normalization methods for the combination coefficients, which correspond to different restrictions imposed on the search space. In the literature of PINNs, the recent work [46] proposes the Kronecker neural network (KNN) based on adaptive linear combinations of candidate activation functions and applies it to solve PDEs. However, only one type of adaptive activation function is discussed in their work under the context of PINNs, and the designs

of its candidate function set and normalization method of combination coefficients are yet to be explored to better suit the optimization of PINNs.

Since most existing works mainly focus on convolutional neural networks for the image classification task, obstacles exist in their direct application in the context of PINNs, which have a higher demand for the smoothness and diversity of the learned functions. First, the optimization of PDE-based constraints needs the activation function to provide higher-order derivatives, which causes the failure of widely-used ReLUs and other piecewise linear functions [35–37] in PINNs. Second, unlike the image classification tasks, different PDE systems could have distinct characteristics, such as periodicity and rapid decay. This leads to a higher requirement for the diversity of the learned functions.

To avoid the inefficient manual selection of activation functions and to alleviate the optimization difficulty of PINNs, we propose the Adaptive Blending Units for PINNs (ABU-PINN) to combine the simple idea of learning combinations of candidate activation functions with special treatments tailored for PINN optimizations. We first remove those activation functions which cannot provide continuous and non-zero higher-order derivatives from the candidate function set. To deal with the various characteristics of PDEs, we incorporate elementary functions with different properties as candidate functions according to our prior knowledge of a physics system at hand. For instance, we can leverage the sinusoidal and exponential functions to reduce the modeling difficulty of periodicity and rapid decay phenomena, which are commonly found in physical systems. Note that we reveals that adaptive activation functions can be more powerful and interpretable in handling PDE systems than they are in general domains like image classification or natural language processing [42–45], because there is no such prior knowledge to leverage. Besides these elementary functions, we also add most of commonly-used activation functions to ensure the diversity of the candidate set. Finally, we further enhance the search space with adaptive slopes [47,48]. The adaptive slopes are proposed to improve the convergence rate of PINNs, but they cannot avoid the manually selection of activation function when solving different PDEs. We show that our method is orthogonal to these methods and can be extended by introducing the adaptive slope into each function in the candidate set.

Our work focuses on the influence of adaptive activation functions on PINNs optimization. The specific contributions of this paper can be summarized as:

- We shed light on the relationship between activation functions and the optimization difficulty of PDE-based constraint. Specifically, we highlight the sensitivity of PINNs to the choice of activation functions, which can be related to various characteristics of the PDE systems. We hope our work could inspire further study on the convergence issue of PINNs from the perspective of network architectures.
- We explore the automatic design of activation functions for PINNs. We evaluate different adaptive activation functions on a series of PDEs, including Poisson's equation, convection equation, Burgers' equation, Allen-Cahn equation, Korteweg-de

Vries equation, Cahn-Hilliard equation and Navier-Stokes equations. The limitations of existing methods are discussed in the context of PINNs.

- We tailor the simple idea of learning combinations of activation functions to the optimization of PINNs. The resulting method can be used to solve different types of PDE systems in an interpretable way. Extensive experiments demonstrate the effectiveness of the proposed method. We also present a comprehensive ablation study to provide a better understanding. We further attempt to explain the effectiveness of learning combinations of activation functions from the perspective of neural tangent kernel.

The paper is organized as follows. In Section 2, we first give a brief introduction to physics-informed neural networks. Then, we investigate the influence of activation functions on PINNs on a simple benchmark problem with analytical solutions in Section 2. The results suggest that the choice of activation functions is crucial for PINNs and depends on the problem. Motivated by this observation, we propose to learn specialized activation functions for different PDE systems with adaptive activation functions as described in Section 2. Subsequently, we compare different types of adaptive activation functions through extensive experiments in Section 3. Finally, we summarize our work and discuss the limitations in Section 4.

2 Method

Physics-informed neural networks

Physics-informed neural networks have emerged as a promising method for solving forward and inverse problems of integer-order PDEs [1, 2, 4, 5], fractional PDEs [49], and stochastic PDEs [50]. In this work, we consider PDEs of the general form

$$\mathbf{u}_t + \mathcal{N}[\mathbf{u}(t, \mathbf{x}); \lambda] = 0, \quad \mathbf{x} \in \Omega \subset \mathbb{R}^d, \quad t \in [0, T], \quad (2.1)$$

subject to the initial and boundary conditions

$$\mathbf{u}(0, \mathbf{x}) = \mathbf{u}^0(\mathbf{x}), \quad \mathbf{x} \in \Omega, \quad (2.2)$$

$$\mathcal{B}[\mathbf{u}] = 0, \quad \mathbf{x} \in \partial\Omega, \quad t \in [0, T], \quad (2.3)$$

where $\mathbf{u}(t, \mathbf{x})$ denotes the solution, $\mathcal{N}[\cdot; \lambda]$ is a differential operator parameterized by λ , $\mathcal{B}[\cdot]$ is a boundary operator, and subscripts denote the partial differentiation.

A physics-informed neural network (PINN) $\mathbf{u}'(t, \mathbf{x}; \theta)$ is optimized to approximate the solution $\mathbf{u}(t, \mathbf{x})$ by minimizing the following objective function

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{ic}}(\theta) + \mathcal{L}_{\text{bc}}(\theta) + \mathcal{L}_{\text{r}}(\theta), \quad (2.4)$$

where

$$\mathcal{L}_{ic}(\theta) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \|u'(0, x_{ic}^i; \theta) - u^0(x_{ic}^i)\|_2^2, \quad (2.5)$$

$$\mathcal{L}_{bc}(\theta) = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \|\mathcal{B}[u'(t_{bc}^i, x_{bc}^i; \theta)]\|_2^2, \quad (2.6)$$

$$\mathcal{L}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} \|u'_t(t_r^i, x_r^i; \theta) + \mathcal{N}[u'(t_r^i, x_r^i; \theta); \lambda]\|_2^2. \quad (2.7)$$

Here θ denotes the weights and biases of neural networks. The $\mathcal{L}_{ic}$ and $\mathcal{L}_{bc}$ measure the prediction error on initial training data $\{x_{ic}^i\}_i^{N_{ic}}$ and boundary training data $\{t_{bc}^i, x_{bc}^i\}_i^{N_{bc}}$. The residual loss $\mathcal{L}_r$ is imposed to make the neural network satisfy the PDE constraint on a set of collocation points $\{t_r^i, x_r^i\}_i^{N_r}$. To compute the residual loss $\mathcal{L}_r$, partial derivatives of the neural network output with respect to t and x can be obtained via automatic differentiation techniques [51].

Activation functions in PINNs

Across the deep learning community, the ReLU enjoys widespread adoption and becomes the default activation function. However, there is no such default choice of activation functions for PINNs, regardless of the importance of activation functions in the optimization and expressivity of neural networks. One of the reasons lies in the failure of ReLUs in PINNs, whose second-order derivative is zero everywhere. More importantly, PINNs show great sensitivity to activation functions due to the various characteristics of the underlying PDE system. To give a further illustration, we compare the performance of PINNs with different activation functions on the following toy examples.

Problem formulation Here we consider the 1D Poisson's equation as

$$\Delta u(x) = f(x), \quad x \in [0, \pi], \quad (2.8)$$

with boundary condition $u(0)=0$ and $u(\pi)=\pi$, where Δ is the Laplace operator and $f(x)$ denotes the source term. As shown in Table 1, we use different source terms to construct analytical solutions with various characteristics. For the problem **P1**, the solution is composed of trigonometric functions with different periods; for the problem **P2**, its solution is dominant by the exponential function. Following the setting described in [13], we impose the boundary conditions as hard-constraints by choosing the surrogate of solution as

$$\hat{u}(x) = x(x - \pi)u'(x; \theta) + x, \quad (2.9)$$

where $u'(x; \theta)$ is the output of PINN. Hence, the training objective is reduced to the residual loss $\mathcal{L}_r$. Note that we consider an extreme case of insufficient collocation points to highlight the difference between activation functions.

Table 1: The source terms and analytical solutions for the 1D Poisson's equation.

Problems	$f(x)$	$u(x)$
P1	$-(\sum_{i=1}^{20} i \sin(ix))$	$x + \sum_{i=1}^{20} \sin(ix)/i$
P2	$(x^3 - 2\pi x^2 + (\pi^2 + 3)x - 2\pi)e^{\frac{(x-\pi)^2}{2}}$	$xe^{\frac{(x-\pi)^2}{2}}$

For each problem, we use a 4-layer fully-connected networks with the hidden dimension set to 16. The model is initialized by the Xavier initialization [52] and trained for 20000 iterations using the Adam optimizer [53]. We initialize the learning rate as $1e-3$ and adapt it with a half-cycle cosine decay schedule. We use 32 collocation points for both **P1** and **P2**. For each problem, we experiment with 8 standard activation functions as described in Appendix A, including trigonometric functions (sin and cos), the hyperbolic tangent function (tanh), the logistic function (sigmoid), the Softplus function (Softplus) [54,55], the Exponential Linear Unit (ELU) [56], the Gaussian Error Linear Unit (GELU) [57] and the Swish function (Swish) [34, 57, 58]. We repeat each experiment 10 times and report the mean and standard deviation of the L_2 relative error.

The choice of activation functions affects the optimization of PINNs. Table 2 shows the results of different activation functions on **P1** and **P2**. As we can see, the choice of activation functions makes a big difference on the prediction accuracy. For example, there exists a large performance gap between the exp and tanh functions on **P2** ($0.73 \pm 2.08\%$ vs. $75.77 \pm 7.93\%$). In Fig. 1, we visualize the second-order derivative of PINNs to analyse the influence of activation functions on the optimization of residual loss $\mathcal{L}_r$. We find that the commonly-used tanh function suffers from severe overfitting when solving these two problems with insufficient collocation points. In contrast, the optimization difficulty can be significantly reduced under the same conditions by using appropriate functions, such as the sin function for **P1** and the exp function for **P2**.

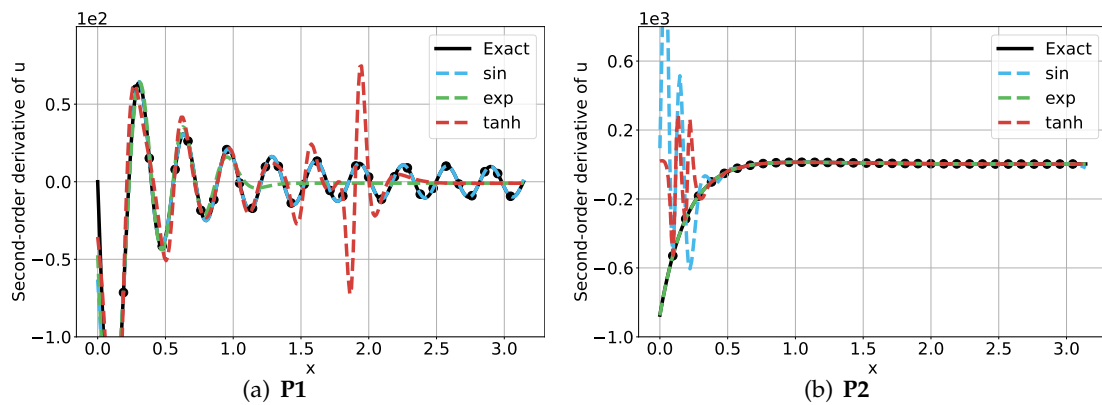
The optimal activation function depends on the property of PDE system. As we can see in Table 2, the sin and exp functions achieve the best performance to model the periodic nature of **P1** and the rapid decay of **P2**, respectively. This is because they can embed similar properties into the neural networks. Note that neither of them could achieve the best performance simultaneously on these two problems, due to the difference in the characteristics of the problems. One can find that while sin achieves the lowest error on **P1**, it produces poor results on **P2**. A similar phenomenon is observed on the exp function. This finding demonstrates the necessity of careful selection of activation functions when solving different problems.

Learning specialized activation functions for solving PDEs

In light of the diversity and complexity of PDEs, it is critical to select proper activation functions in PINNs. Existing works usually make the choice though trial-and-error.

Table 2: Comparisons between different activation functions on 1D Poisson's equation. L_2 relative error (%) is reported.

Problems	sin	exp	tanh	sigmoid
P1	0.91 ± 0.30	39.11 ± 31.32	34.27 ± 45.21	23.51 ± 17.82
P2	49.42 ± 4.75	0.73 ± 2.08	75.77 ± 7.93	56.36 ± 10.81
Problems	Softplus	ELU	GELU	Swish
P1	86.03 ± 64.79	36.68 ± 20.61	19.48 ± 18.50	33.16 ± 36.81
P2	0.65 ± 0.23	40.96 ± 97.75	2.70 ± 3.41	1.05 ± 0.42

Figure 1: Visualization of the residual loss $\mathcal{L}_r$. In the case of 1D Poisson's equation, the residual loss $\mathcal{L}_r$ penalizes the deviation of the second-order derivative of PINNs to the exact values on sampled collocation points (black dots in figures). We plot the results of sin, exp and tanh.

However, this strategy is inefficient especially when solving complex problems, where searching for a set of activation functions is necessary to make accurate predictions. For instance, a combination of the sinusoidal and the exponential function is demonstrated effective to solve the heat transfer equation, whose solution is periodic in space and exponentially decaying in time [59]. In this case, the trial-and-error strategy leads to the combinatorial search problem, which becomes infeasible when the candidate activation function set is large. To address this problem, we introduce adaptive activation functions into PINNs to search for the optimal functions for different problems automatically.

As we mentioned before, the major difference of existing adaptive activation functions lies in their search space. Here we consider four types of search space which could provide higher-order derivatives. Their formulations are as follows.

- **Linear combinations of common activation functions** [42–45]: instead of categorically selecting one particular activation function, this kind of methods constructs the search space to be a linear combination of all candidate activation functions with learnable coefficients. We use the formulation from the Adaptive Blending

Unit (ABU) [45], because it covers different normalization methods for the coefficients. This formulation is presented as:

$$f(x) = \sum_{i=1}^N G(\alpha_i) \sigma_i(x), \quad (2.10)$$

where $\sigma_i(\cdot)$ and α_i denote a candidate activation function and a learnable parameter, respectively. Together with a gate function $G(\cdot)$, the parameter α_i determines the weight (or coefficient) of its corresponding candidate activation function. Different gate functions correspond to different normalization methods in ABU. By default, we use the softmax function $G(\alpha_i) = \exp(\alpha_i) / \sum_{j=1}^N \exp(\alpha_j)$. In this case, the search space is restricted to the convex hull of the set of candidate activation functions.

- **Learnable functions based on Taylor approximation:** the SLAF [38] proposes to approximate the optimal activation functions for different tasks with Taylor polynomial bases, which is presented as:

$$f(x) = \sum_{i=0}^m a_i x^i, \quad (2.11)$$

where m is the order of Taylor series and is set to 5 in our experiments. The search space of SLAF covers a wide range of existing activation functions which could be accurately approximated by Taylor polynomial bases. The learnable parameters $\{a_i\}_{i=0}^m$ are initialized from a normal distribution.

- **Learnable functions based on Padé approximation:** Padé Activation Units (PAU) adopt the Padé approximation to construct its search space [39], which is presented as rational functions of the form

$$f(x) = \frac{\sum_{i=0}^m a_i x^i}{1 + |\sum_{j=1}^n b_j x^j|}, \quad (2.12)$$

where m is set to 5 and n is set to 4 following the default setting in [39]. As an alternative to Taylor approximation, the Padé approximant often provides a better approximation of a given activation function. Note that the absolute value in its formulation is to avoid a zero-valued denominator. The learnable parameters $\{a_i\}_{i=0}^m$ and $\{b_j\}_{j=1}^n$ are initialized from a normal distribution.

- **Approximators for the general Maxout family:** motivated by the fact that the Swish function is a smooth approximation to ReLU, the ACON [40] is proposed as an approximator to the general Maxout family activation functions. Its formulation is presented as

$$f(x) = (p_1 - p_2)x * \text{sigmoid}(\beta(p_1 - p_2)x) + p_2x, \quad (2.13)$$

where p_1 , p_2 and β are learnable parameters. It is a smooth approximator to the $\max(p_1x, p_2x)$ function. By fixing $p_1 = 1$ and $p_2 = 0$, the ACON is reduced to the Swish function. Compared with other adaptive activation functions, the search space of ACON has a limited degree of flexibility. The learnable parameters p_1 , p_2 and β are initialized from a normal distribution.

Since previous works mainly focus on the convolutional neural networks for the image classification task, learning adaptive activation functions in the context of PINNs faces new challenges in terms of smoothness and diversity, which arises from the peculiar nature of the problem. For example, the polynomial-based SLAF is subject to the exploding values and oscillation problems, which may hinder the optimization of PDE-based constraints. We also find that the PAU suffers from the training instability, because its higher-order derivatives are not continuous. While ACON meets the requirement of smoothness, its search space has limited diversity, which limits its performance in solving different PDEs with various characteristics.

To remove obstacles to apply adaptive activation functions to PINNs, we propose to tailor the idea of learning combinations of activation functions to the optimization of PDE-based constraints. Taking the ABU [45] as an example, its search space is built upon the candidate functions set $\mathcal{F} = \{\sigma_1, \sigma_2, \dots, \sigma_N\}$. First, we remove those activation functions which cannot provide continuous and non-zero higher-order derivatives, such as ReLU, ELU and identity function. Second, we incorporate elementary functions with different properties according to our prior knowledge of a physics system at hand. For example, the sinusoidal and exponential functions can be leveraged to model the periodicity and rapid decay phenomena, which are difficult to model by neural networks with normal activation functions such as tanh as shown in Section 2. Note that none of the existing works [42–45] have considered adding these functions to their candidate set, because there is no such prior knowledge to leverage in general domains like image classification or natural language processing. In contrast, we reveal that adaptive activation functions can be more powerful and interpretable in handling PDE systems. In addition to these elementary functions, we also include most of commonly-used activation functions to ensure the diversity. Finally, we further enhance the search space with the adaptive slope, which is proposed to improve the convergence rate of standard activation functions for PINNs [47,48]. This can be achieved by simply introducing a learnable scaling factor β_i for each candidate function σ_i , which can be represented as

$$f(x) = \sum_{i=1}^N G(\alpha_i) \sigma_i(\beta_i x). \quad (2.14)$$

We denote the modified version of ABU as ABU-PINN to highlight the special treatment tailored for PINNs.

With adaptive activation functions, we formulate the manual selection of activation functions as a learning problem. We employ these methods by replacing all standard activation functions in PINNs. They can be applied in a neuron-wise manner where learn-

able parameters are allowed to be vary across neurons, or in a layer-wise manner where the parameters are shared for all neurons in one layer. Their learnable parameters can be optimized with other weights and biases in the networks through backpropagation to minimize the loss functions. Therefore, the neural networks learn to adapt the shape of activation functions to the PDE system during the training process. Besides the efficient optimization, the continuous parameterization of search space enables the discovery of novel functions which are different from the hand-designed ones.

3 Experiments

In this section, we first revisit the 1D Poisson's equation to show ABU-PINN can solve different types of problems in an interpretable way in Section 3. Next, we compare the performance of different adaptive activation functions on several benchmarks of 1D time-dependent PDEs in Section 3, as well as 2D PDEs in Section 3. Then, Section 3 provides detailed ablation results to analyse the proposed ABU-PINN. Finally, in Section 3, we present an intuitive interpretation of the performance gain of ABU-PINN from the neural tangent kernel perspective.

Experiments on 1D Poisson's equation

First, we use the 1D Poisson's equation with different source terms as described in Section 2 to demonstrate that the proposed ABU-PINN could learn optimal activation functions for problems with different properties by incorporating the prior knowledge of the PDE systems. To advance the modeling of the periodic nature of **P1** and the rapid decay of **P2** simultaneously, we add the sin and exp into the candidate functions to reduce the training difficulty of PINNs. We define $\mathcal{F} = \{\sin, \exp\}$ for simplicity and expect ABU-PINN could adapt its attention over these two candidate functions to the problem. The trainable parameters $\{\alpha_{\sin}, \alpha_{\exp}\}$ are initialized as zeros, which implies equal attention over all candidate functions. We follow the experiments setting described in Section 2, except that we replace all the fixed-shape activation functions as ABU-PINN.

The results of ABU-PINN and its two candidate functions are presented in Table 3(a). It can be observed that ABU-PINN can achieve comparable accuracy with the sin on **P1** and outperform the exp on **P2**, despite the large performance gap between its two candidate functions. The results verify the effectiveness of ABU-PINN for solving PDE systems with different source terms by incorporating suitable candidate functions. We also report the values of learned coefficients in Table 3(b). For **P1**, ABU-PINN pays more attention on the sin function as expected. For **P2**, we observe that the learned activation functions are different across layers. The exp is preferred only on the third layer, which still leads a performance gain compared with standard activation functions. The results indicate that ABU-PINN can learn useful combination coefficients for different problems.

Table 3: Comparisons of standard activation and ABU-PINN on the 1D Poisson's equation. L_2 relative error (%) is reported. We also report the learned coefficients $\alpha_{\sin}$ and $\alpha_{\exp}$.

(a) The L_2 relative error (%)

Problems	sin	exp	ABU-PINN
P1	0.91 ± 0.30	39.11 ± 32.32	0.88 ± 0.42
P2	49.42 ± 4.75	0.73 ± 2.08	0.01 ± 0.006

(b) The learned coefficients

Problems	Layer-1		Layer-2		Layer-3	
	$\alpha_{\sin}$	$\alpha_{\exp}$	$\alpha_{\sin}$	$\alpha_{\exp}$	$\alpha_{\sin}$	$\alpha_{\exp}$
P1	0.81 ± 0.02	0.19 ± 0.02	0.75 ± 0.06	0.25 ± 0.06	0.84 ± 0.04	0.16 ± 0.04
P2	0.58 ± 0.08	0.42 ± 0.08	0.53 ± 0.05	0.47 ± 0.05	0.45 ± 0.06	0.55 ± 0.06

Experiments on 1D time-dependent PDEs

We provide a comprehensive comparison between different adaptive activation functions to solve various time-dependent PDEs, ranging from first-order linear PDE to fourth-order nonlinear PDE, including the convection equation, the Burgers' equation, the Allen-Cahn equation, the Korteweg-de Vries equation, and the Cahn-Hilliard equation.

Problem formulations

Convection equation. Consider a one-dimensional linear convection problem as

$$u_t + \beta u_x = 0, \quad x \in [0, 2\pi], \quad t \in [0, 1], \quad (3.1)$$

$$u(0, x) = \sin(x), \quad (3.2)$$

$$u(t, 0) = u(t, 2\pi). \quad (3.3)$$

The solution of this problem is periodic over time, whose period is inversely proportional to the convection coefficient β . The previous work [10] finds it difficult for vanilla PINNs to learn the solution with a large β and proposes to train the PINNs with curriculum learning. We show that accurate prediction can be achieved with a normal training strategy by using a suitable activation function. Moreover, the performance can be further improved by the proposed ABU-PINN. Here we set the convection coefficient β to 64. We use $N_{ic} = 512$ points for the initial condition and $N_{bc} = 200$ for the boundary condition. The residual loss $\mathcal{L}_r$ is computed on $N_r = 6400$ randomly sampled collocation points.

Burgers' equation. We consider the one-dimensional Burgers' equation:

$$u_t + uu_x = \nu u_{xx}, \quad x \in [-1, 1], \quad t \in [0, 1], \quad (3.4)$$

$$u(0, x) = -\sin(\pi x), \quad (3.5)$$

$$u(t, -1) = u(t, 1) = 0, \quad (3.6)$$

where $\nu = 0.01/\pi$. Following [17], we uniformly sample $N_{ic} = 256$ and $N_{bc} = 100$ points for initial and boundary training data, respectively. We compute the residual loss $\mathcal{L}_r$ on $N_r = 4800$ collocation points, which are randomly selected from the spatial-temporal domain.

Allen-Cahn equation. We next consider the Allen-Cahn equation as

$$u_t = Du_{xx} + 5(u - u^3), \quad x \in [-1, 1], \quad t \in [0, 1], \quad (3.7)$$

$$u(0, x) = x^2 \cos(\pi x), \quad (3.8)$$

$$u(t, -1) = u(t, 1) = -1, \quad (3.9)$$

where $D = 0.001$ following the setting in [13]. We choose the following surrogate of solution to enforce the initial and boundary conditions:

$$\hat{u}(t, x) = x^2 \cos(\pi x) + t(1 - x^2)u'(t, x; \theta), \quad (3.10)$$

where $u'(t, x; \theta)$ is the output of neural networks. We use $N_r = 8000$ collocation points.

Korteweg-de Vries equation. We consider the Korteweg-de Vries (KdV) equation as

$$u_t + \lambda_1 uu_x + \lambda_2 u_{xxx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1], \quad (3.11)$$

$$u(0, x) = \cos(\pi x), \quad (3.12)$$

$$u(t, -1) = u(t, 1), \quad u_x(t, -1) = u_x(t, 1), \quad (3.13)$$

where $\lambda_1 = 1$ and $\lambda_2 = 0.0025$ following the setting in [1]. We choose the following surrogate of solution to enforce the initial conditions:

$$\hat{u}(t, x) = \cos(\pi x) + tu'(t, x; \theta), \quad (3.14)$$

where $u'(t, x; \theta)$ is the output of neural networks. We use $N_{bc} = 200$ training points for the boundary condition and $N_r = 8000$ collocation points for the residual loss $\mathcal{L}_r$.

Cahn-Hilliard equation. We consider the phase space representation of Cahn-Hilliard equation as

$$u_t - \nabla^2(-\lambda_1 \lambda_2 h + \lambda_2(u^3 - u)) = 0, \quad h = \nabla^2 u, \quad x \in [-1, 1], \quad t \in [0, 1], \quad (3.15)$$

$$u(0, x) = \cos(\pi x) - \exp(-4(\pi x)^2), \quad (3.16)$$

$$u(t, -1) = u(t, 1), \quad u_x(t, -1) = u_x(t, 1), \quad (3.17)$$

$$h(t, -1) = h(t, 1), \quad h_x(t, -1) = h_x(t, 1), \quad (3.18)$$

where $\lambda_1 = 0.02$ and $\lambda_2 = 1$ following the setting in [60]. The previous work proposes a novel sequential training method to solve this strongly non-linear and high-order problem [60]. We find that a normal training strategy can lead to a comparable result if we add more collocation points around $t = 0$. To be specific, we sample 4000 collocation points from time interval $[0, 0.05)$ and 8000 points from $[0.05, 1]$. We set $N_{ic} = 256$ and $N_{bc} = 100$. We show that PINNs under this training setting can work as a strong baseline and can be further improved by ABU-PINN.

Main results

Experiment setups. We train the PINNs by a two-stage optimization except the convection equation. At the first stage, the model is trained for a certain number of iterations by Adam [53]. Then, the L-BFGS [61] is used to train the network until convergence. The first stage is to provide a good initialization for the L-BFGS optimizer. In the case of convection equation, only the Adam optimizer is used. The detailed experimental configurations can be found in Table 4. The model is initialized by the Xavier initialization. The learning rate is adapted with a half-cycle cosine decay schedule in the first stage with Adam optimizer. We repeat each experiment 5 times and report the mean and standard deviation of the L_2 relative error. We run all experiments on one NVIDIA GeForce RTX 2080Ti GPU.

Table 4: The experimental configurations for 1D time-dependent problems.

Problems	Network		Adam		L-BFGS	Data		
	Depth	Width	Iters	lr	Max Iters	N_{ic}	N_{bc}	N_r
Convection equation	6	64	100k	2e-3	-	512	200	6400
Burgers' equation	4	32	15k	1e-3	15k	256	100	5300
Allen-Cahn equation	4	32	40k	1e-3	15k	-	-	8000
KdV equation	4	32	40k	1e-3	15k	-	200	8000
Cahn-Hilliard equation	4	32	100k	1e-3	15k	256	100	15000

Setups of standard and adaptive activation functions. We experiment with several commonly-used activation functions, including the sinusoidal functions (sin), the hyperbolic tangent function (tanh), the logistic function (sigmoid), the Softplus function (Softplus) [54,55], the Exponential Linear Unit (ELU) [56], the Gaussian Error Linear Unit (GELU) [57] and the Swish function (Swish) [34]. The details of each activation function can refer to Appendix A. We compare the performance of adaptive activation functions which could provide higher-order derivatives, including SLAF, PAU, ACON and ABU-PINN. We employ all adaptive activation functions in a layer-wise manner by default. For SLAF, PAU and ACON, the learnable parameters are initialized from a normal distribution. We set the candidate set $\mathcal{F}$ as $\{\sin, \tanh, \text{GELU}, \text{Swish}, \text{Softplus}\}$ and initialize

Table 5: Comparisons of standard activation functions, adaptive activation functions and their counterparts with adaptive slopes (AS) on 1D time-dependent PDEs. We report the L_2 relative error (%) for each problem and the average error rate over all problems. The better results are **bold-faced**.

Method	Convection equation	Burgers' equation	Allen-Cahn equation	KdV equation	Cahn-Hilliard equation	Average error
sin	0.36 ± 0.15	5.18 ± 3.73	3.57 ± 0.64	0.63 ± 0.17	1.72 ± 0.61	2.29
tanh	6.83 ± 4.79	0.26 ± 0.13	1.34 ± 0.54	1.32 ± 1.12	4.02 ± 4.56	2.75
sigmoid	70.38 ± 2.98	1.24 ± 1.05	1.63 ± 0.13	2.34 ± 0.53	3.12 ± 2.72	15.74
GELU	39.29 ± 35.51	4.43 ± 2.87	3.93 ± 0.78	1.21 ± 0.41	1.01 ± 1.27	9.97
Swish	5.59 ± 2.18	8.27 ± 4.35	5.56 ± 1.28	1.73 ± 0.10	2.22 ± 2.60	4.67
Softplus	55.39 ± 2.46	17.75 ± 8.11	17.72 ± 6.04	6.23 ± 0.44	9.67 ± 2.57	21.35
ELU	6.67 ± 0.96	46.34 ± 2.36	52.55 ± 2.95	78.95 ± 2.57	90.77 ± 2.07	55.66
SLAF	0.36 ± 0.18	43.93 ± 0.66	33.93 ± 9.97	25.23 ± 1.28	52.57 ± 27.93	31.20
PAU	45.78 ± 35.47	48.31 ± 8.21	43.83 ± 15.18	68.11 ± 13.49	115.59 ± 3.79	64.32
ACON	3.55 ± 1.66	1.18 ± 1.55	3.88 ± 1.82	1.52 ± 0.35	2.46 ± 1.96	2.52
ABU-PINN	0.06 ± 0.03	0.21 ± 0.11	0.76 ± 0.26	0.34 ± 0.05	0.50 ± 0.15	0.37
With AS						
sin	0.28 ± 0.08	1.82 ± 1.58	3.57 ± 0.46	0.57 ± 0.19	1.73 ± 0.73	1.59
tanh	3.06 ± 1.65	0.16 ± 0.09	0.81 ± 0.21	1.71 ± 1.53	2.11 ± 0.49	1.57
GELU	38.00 ± 39.53	0.71 ± 0.38	1.99 ± 0.63	0.79 ± 0.17	0.96 ± 0.57	8.49
ABU-PINN	0.05 ± 0.02	0.15 ± 0.10	0.58 ± 0.15	0.34 ± 0.08	0.35 ± 0.07	0.29

the learnable parameters $\{\alpha_i\}_{i=1}^N$ as zeros for ABU-PINN. This candidate set includes the sin function to help the learning of periodicity and several commonly-used activation functions to ensure the diversity. We show that this candidate set achieves robust and consistent performances over these five time-dependent PDEs. We also compare ABU-PINN with standard activation functions with the layer-wise adaptive slopes [48]. In this case, the scaling factors $\{\beta_i\}_{i=1}^N$ are initialized as ones.

Comparisons with fixed activation functions. In Table 5, we presents the L_2 relative error of ABU-PINN and seven standard activation functions on five time-dependent PDEs. One can observe that ABU-PINN outperforms standard activation functions on all PDEs. For example, ABU-PINN reduces the error rate by 83% on the convection equation ($0.06 \pm 0.03\%$ vs. $0.36 \pm 0.15\%$), by 46% on the KdV equation ($0.34 \pm 0.05\%$ vs. $0.64 \pm 0.34\%$), and by 56% on the Cahn-Hilliard equation ($0.50 \pm 0.15\%$ vs. $1.01 \pm 1.27\%$), compared with optimal standard activation function of each problem. More importantly, ABU-PINN performs consistently over all these problems, while standard activation functions suffer from high variance in the performance of different PDEs. We present the predicted solutions of ABU-PINN for convection equation in Fig. 2 and Allen-Cahn equation in Fig. 3. More visualization results can be found in Figs. 10-12 in the Appendix.

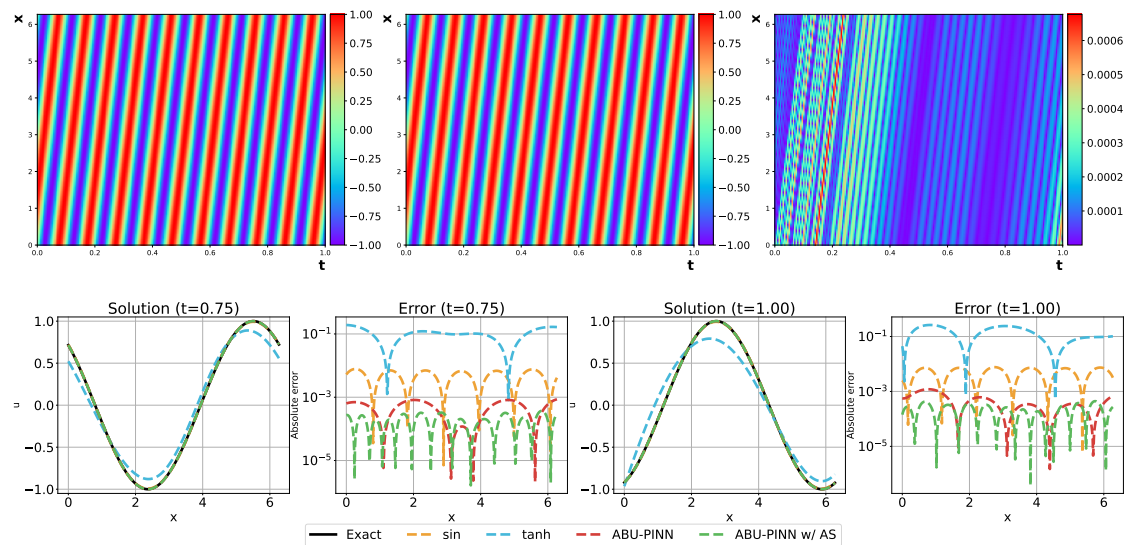


Figure 2: The convection equation. Top: the exact solution (left), the predictions of ABU-PINN (middle), and the absolute error between them (right). Bottom: predicted solutions at different time snapshots.

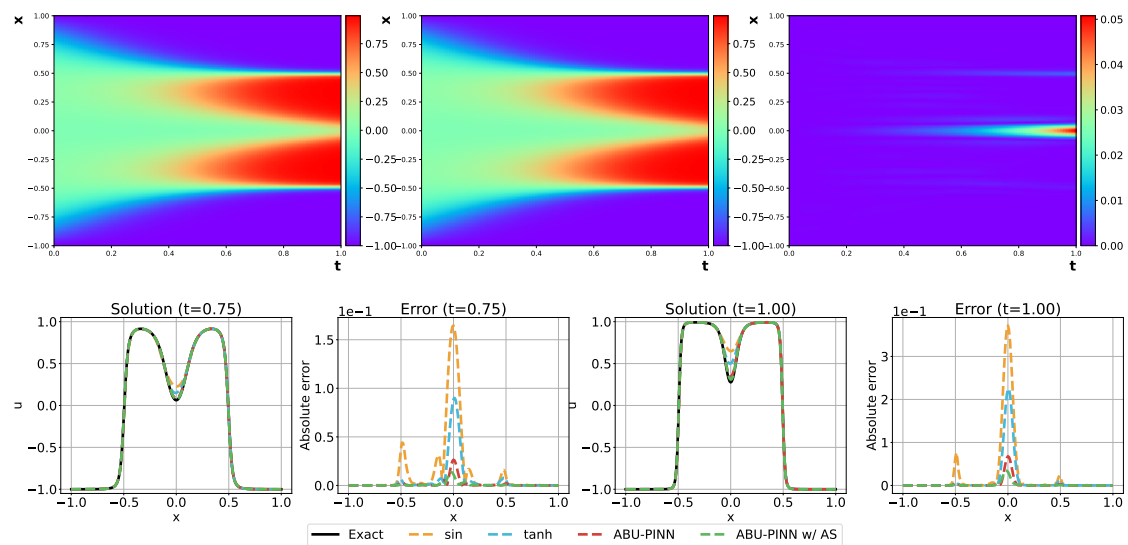


Figure 3: The Allen-Cahn equation. Top: the exact solution (left), the predictions of ABU-PINN (middle), and the absolute error between them (right). Bottom: predicted solutions at different time snapshots.

Comparisons with other adaptive activation functions. The results of different adaptive activation functions are shown in Table 5. SLAF achieves performance comparable to the best standard activation function on the first-order convection equation ($0.36 \pm 0.18\%$ vs. $0.36 \pm 0.15\%$), but does not produce accurate predictions for other higher-order PDEs.

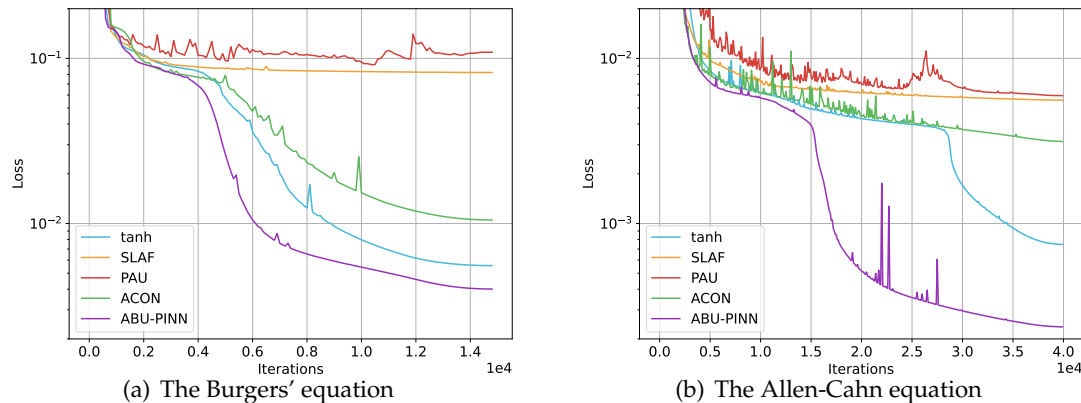


Figure 4: The training losses of different activation functions against iterations.

Although SLAF shares a similar formulation with ABU-PINN, its polynomial bases can cause vanishing or exploding gradients due to the high order powers in its derivatives. This problem might hinder the optimization of higher-order PDE-based constraints. One can observe that PAU performs poorly on all PDEs. To figure out the reason, we plot the training loss in Fig. 4 and find PAU suffers from instability of training. We attribute the training instability of PAU to its discontinuous derivatives, which arise from the absolute value in the denominator of its formulation. ACON achieves stable performance but does not surpass the best standard activation function in each problem due to the limited flexibility of its formulation. The proposed ABU-PINN surpasses these methods by leveraging its diverse and smooth candidate functions.

Comparisons with activation functions with adaptive slopes. We also compare ABU-PINN with three standard activation function enhanced with adaptive slopes (AS). We find ABU-PINN without AS achieves competitive or even better results. For example, ABU-PINN without AS outperforms the tanh with AS on the Allen-Cahn equation ($0.76 \pm 0.26\%$ vs. $0.81 \pm 0.21\%$) and outperforms the sin with AS on the convection equation ($0.06 \pm 0.03\%$ vs. $0.28 \pm 0.08\%$). The performance of ABU-PINN can be further improved by incorporating adaptive slopes into the search space. As a showcase, ABU-PINN with AS obtains a performance gain by 64% compared to the GELU with AS on the Cahn-Hilliard equation and by 82% compared to the sin with AS on the convection equation. Furthermore, our method has a better generalization ability and achieves the lowest average error rate over all problems (0.29% with AS and 0.37% without AS), while the adaptive slopes cannot alleviate the need of manual selections of activation functions.

Computational cost. We present the computational cost of each activation function in Table 6 and Fig. 13 in the Appendix. The computational cost of ABU-PINN is 3.1 times that of standard activation functions and 1.5 times that of ACON. Despite the incurred

Table 6: The computational cost of each activation function. We report the wall-clock time required to train each network for 10000 iterations in the case of Allen-Cahn equation, conducted on a NVIDIA GeForce RTX 2080Ti GPU.

Method	tanh	SLAF	PAU	ACON	ABU-PINN
Time (sec)	250	720	960	520	770

overhead, the proposed ABU-PINN attains a significant improvement (approximately 85%) in the averaged performance of five one-dimensional time-dependent PDEs. Moreover, ABU-PINN surpasses the other two adaptive activation functions, namely SLAF and PAU, while simultaneously maintaining a comparable computational cost.

Visualization of the learned ABU-PINN. We show the learned ABU-PINN in Fig. 5. One can observe the learned activation functions are different from candidate functions. We also find differences in learned functions of different layers. For example, a deeper layer tends to have a larger weight of sin function in the case of convection equation. Moreover, the learned functions vary across problems, which conforms to our motivation to learn specialized activation functions for different PDE systems.

Experiments on 2D PDEs

Problem formulations.

This section evaluates the efficacy of ABU-PINN on two well-established benchmark problems in the field of computational fluid dynamics, which are governed by the two-dimensional incompressible Navier-Stokes equations.

Flow in a lid-driven cavity. This case involves the simulation of fluid motion inside a square domain, where the top lid is driven with a constant velocity while the remaining boundaries are considered impermeable and stationary. This setup creates a confined cavity where the flow dynamics are determined by the steady-state Navier-Stokes equations as

$$\lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy}) = 0, \quad (x, y) \in \Omega, \quad (3.19)$$

$$\lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy}) = 0, \quad (x, y) \in \Omega, \quad (3.20)$$

$$u_x + v_y = 0, \quad (x, y) \in \Omega, \quad (3.21)$$

$$u(x, y) = 1, \quad v(x, y) = 0, \quad (x, y) \in \mathcal{B}_0, \quad (3.22)$$

$$u(x, y) = 0, \quad v(x, y) = 0, \quad (x, y) \in \mathcal{B}_1, \quad (3.23)$$

where $u(x, y)$ and $v(x, y)$ denote the x-component and y-component, respectively, of the velocity vector, while $p(x, y)$ is the pressure field. The domain $\Omega = (0, 1) \times (0, 1)$ represents a two-dimensional square cavity, where $\mathcal{B}_0$ denotes the top boundary and $\mathcal{B}_1$ represents

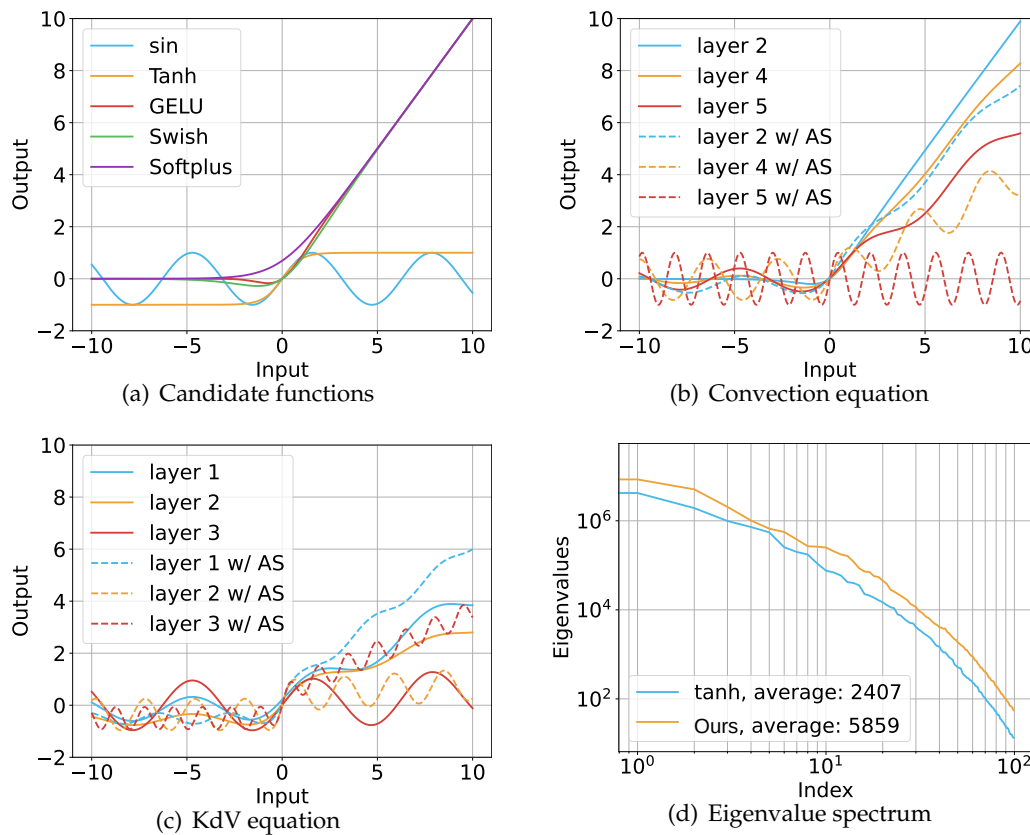


Figure 5: Visualization of the learned activation functions and the eigenvalue spectrum. (a) Candidate functions of ABU-PINN. (bc) The learned activation functions for the convection equation and the KdV equation. The curves of ABU-PINN with and without the adaptive slope are presented. (d) The eigenvalue spectrum of NTK matrix on the training data of the Allen-Cahn equation. The word "Ours" refers to ABU-PINN method.

the other three sides of the cavity. We set $\lambda_1 = 1$ and $\lambda_2 = 0.01$ following the setting in [9]. We use $N_r = 128$ collocation points for the residual loss $\mathcal{L}_r$ and 128 points for each boundary condition (resulting in a total of $N_{bc} = 512$ points). These training points are re-sampled for each iteration.

Flow past a circular cylinder. In this case, we assume a two-dimensional spatial domain of $\Omega = [-15, 25] \times [-8, 8]$, where a circular cylinder with a diameter of $D = 1$ is positioned at the origin. Following the configurations outlined in [1], we consider these boundary conditions: a uniform free stream velocity profile with $u_\infty = 1$ is prescribed at the left boundary, a zero pressure outflow condition is enforced at the right boundary, situated 25 diameters downstream of the cylinder, and a periodic condition is imposed for the top and bottom boundaries. We assume a kinematic viscosity of $\nu = 0.01$, resulting in a corresponding Reynolds number of $Re = u_\infty D / \nu = 100$. The system is governed by

Table 7: The experimental configurations for 2D problems.

Problems	Network		Adam		Data		
	Depth	Width	Iters	lr	N_m	N_{bc}	N_r
Flow in a lid-driven cavity	6	64	80k	1e-3	-	128	128
Flow past a circular cylinder	6	64	40k	1e-3	5000	-	5000

the unsteady-state Navier-Stokes equations as

$$u_t + \lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy}) = 0, \quad (x, y) \in \Omega, \quad t \in [0, T], \quad (3.24)$$

$$u_t + \lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy}) = 0, \quad (x, y) \in \Omega, \quad t \in [0, T], \quad (3.25)$$

$$u_x + v_y = 0, \quad (x, y) \in \Omega, \quad t \in [0, T], \quad (3.26)$$

where λ_1 is set to 1, $\lambda_2 = 1/Re = 0.01$ and $T = 20$. We focus on the solution within the wake region behind the cylinder, which corresponds to the spatial domain denoted as $\Omega' = [1, 8] \times [-2, 2]$. We consider the inverse problem formulation of this case, where λ_1 and λ_2 are assumed to be unknown. Given a limited set of measurements of the velocity fields, denoted as $\{t_m^i, x_m^i, y_m^i, u_m^i, v_m^i\}_i^{N_m}$, the PINNs are optimized to accurately predict the dynamics of this system and estimate the values of λ_1 and λ_2 . The objective function $\mathcal{L}(\theta)$ is composed of the residual loss $\mathcal{L}_r$ in Eq. (2.7) and the data loss $\mathcal{L}_m$

$$\mathcal{L}(\theta) = \mathcal{L}_r(\theta) + \mathcal{L}_m(\theta), \quad (3.27)$$

$$\mathcal{L}_m(\theta) = \frac{1}{N_m} \sum_{i=1}^{N_m} (||u'(t_m^i, x_m^i, y_m^i; \theta) - u_m^i||_2^2 + ||v'(t_m^i, x_m^i, y_m^i; \theta) - v_m^i||_2^2). \quad (3.28)$$

Main results

We replicate the experimental settings outlined in Section 3, maintaining consistency, with the exception that we exclusively employ the Adam optimizer during the training of PINNs. The detailed experimental configurations for 2D PDEs can be found in Table 7. The results are shown in Table 8. The proposed method exhibits superior performance across all evaluated metrics when compared to its candidate activation functions. In the scenario of flow in a lid-driven cavity, the utilization of ABU-PINN yields a notable reduction in the error rates for both the x-component velocity u and the y-component velocity v . Specifically, ABU-PINN achieves a reduction of 11% in the error rate for u and 20% in the error rate for v when compared to the best-performing standard activation function, GELU. In the case of flow past a circular cylinder, the performance of ABU-PINN surpasses that of standard activation functions not only in predicting the dynamics of the system but also in estimating the values of λ_1 and λ_2 . As a demonstration, ABU-PINN achieves a significant performance improvement of 38% and 27% in the estimation of λ_1 and λ_2 , respectively, compared to the GELU activation function. This superior performance of ABU-PINN highlights its enhanced capability in handling inverse problems.

Table 8: Experimental results on 2D PDEs. We report the L_2 relative error (%) for each variable. In the case of flow past a circular cylinder, we present the averaged results obtained from 200 temporal snapshots covering the time interval from $t=0$ to $t=20$, with a time increment of 0.1. The best results are **bold-faced**, while the second best results are underlined.

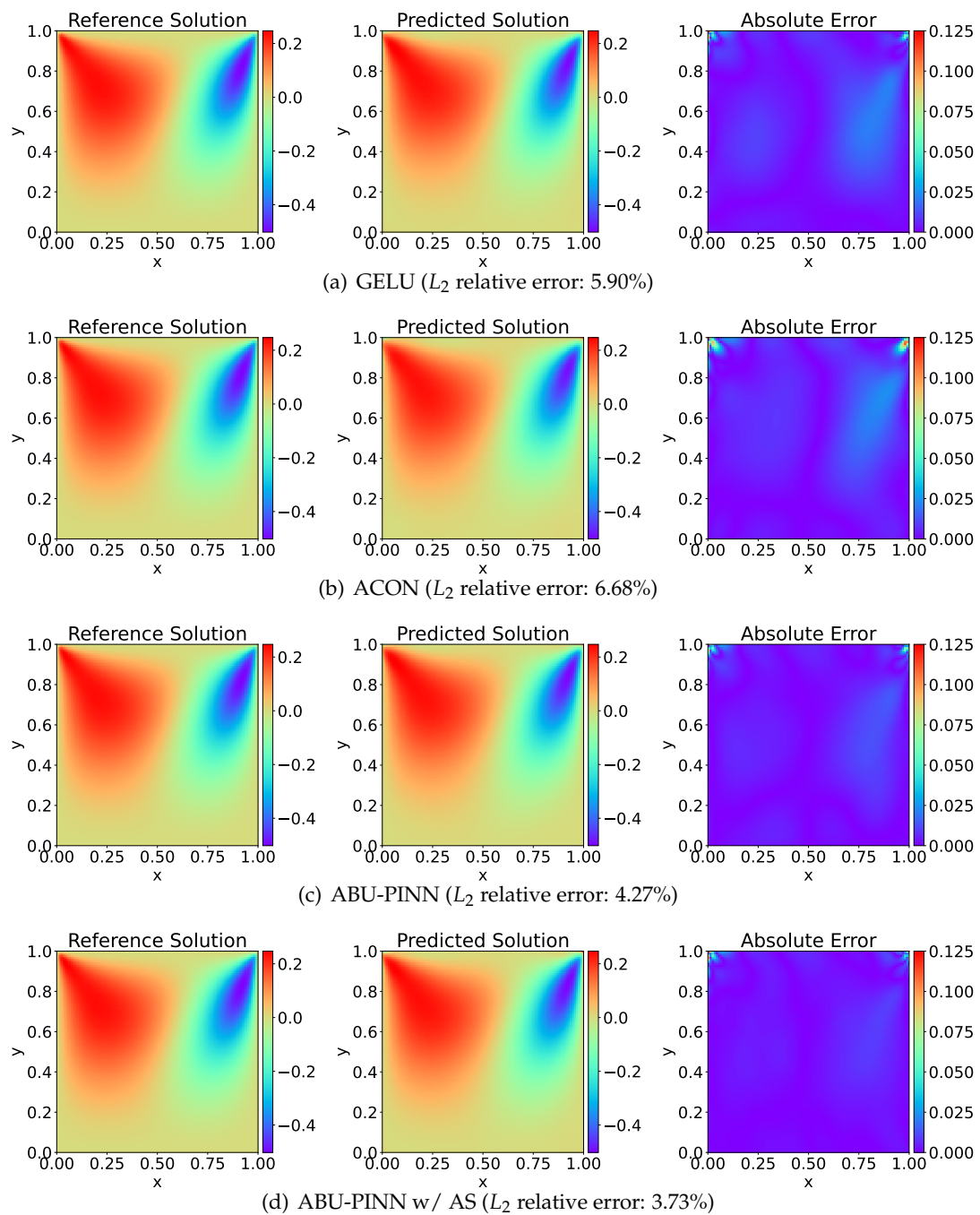
Method	Flow in a lid-driven cavity		Flow past a circular cylinder				
	u	v	u	v	p	λ_1	λ_2
sin	5.36 ± 0.95	7.65 ± 2.08	0.69 ± 0.09	1.97 ± 0.30	2.59 ± 0.39	0.15 ± 0.03	10.44 ± 1.12
tanh	7.65 ± 1.18	11.78 ± 1.98	0.54 ± 0.03	1.70 ± 0.06	2.50 ± 0.13	0.09 ± 0.01	7.53 ± 0.52
GELU	4.48 ± 0.10	5.71 ± 0.11	0.52 ± 0.02	1.45 ± 0.04	2.24 ± 0.05	0.08 ± 0.03	6.83 ± 0.66
Swish	4.90 ± 0.49	7.02 ± 1.15	0.61 ± 0.05	1.68 ± 0.16	2.74 ± 0.08	1.17 ± 0.04	9.29 ± 1.29
Softplus	8.78 ± 0.43	14.63 ± 1.15	1.61 ± 0.12	4.52 ± 0.49	6.76 ± 0.77	0.78 ± 0.10	11.76 ± 0.24
SLAF	4.78 ± 0.09	6.42 ± 0.32	1.39 ± 0.64	3.20 ± 1.47	4.13 ± 1.21	0.34 ± 0.22	13.70 ± 3.01
PAU	87.43 ± 8.78	97.36 ± 3.12	1.24 ± 0.05	3.34 ± 0.12	7.56 ± 0.47	1.11 ± 0.07	19.27 ± 8.00
ACON	4.67 ± 0.17	6.27 ± 0.41	0.70 ± 0.13	1.87 ± 0.36	3.04 ± 0.56	0.10 ± 0.03	6.52 ± 1.79
ABU-PINN	<u>3.99 ± 0.07</u>	<u>4.55 ± 0.28</u>	<u>0.48 ± 0.01</u>	<u>1.35 ± 0.05</u>	<u>2.11 ± 0.02</u>	<u>0.05 ± 0.02</u>	<u>4.97 ± 0.13</u>
ABU-PINN (AS)	3.59 ± 0.09	3.97 ± 0.24	0.38 ± 0.01	1.07 ± 0.01	1.87 ± 0.06	0.02 ± 0.006	4.58 ± 0.11

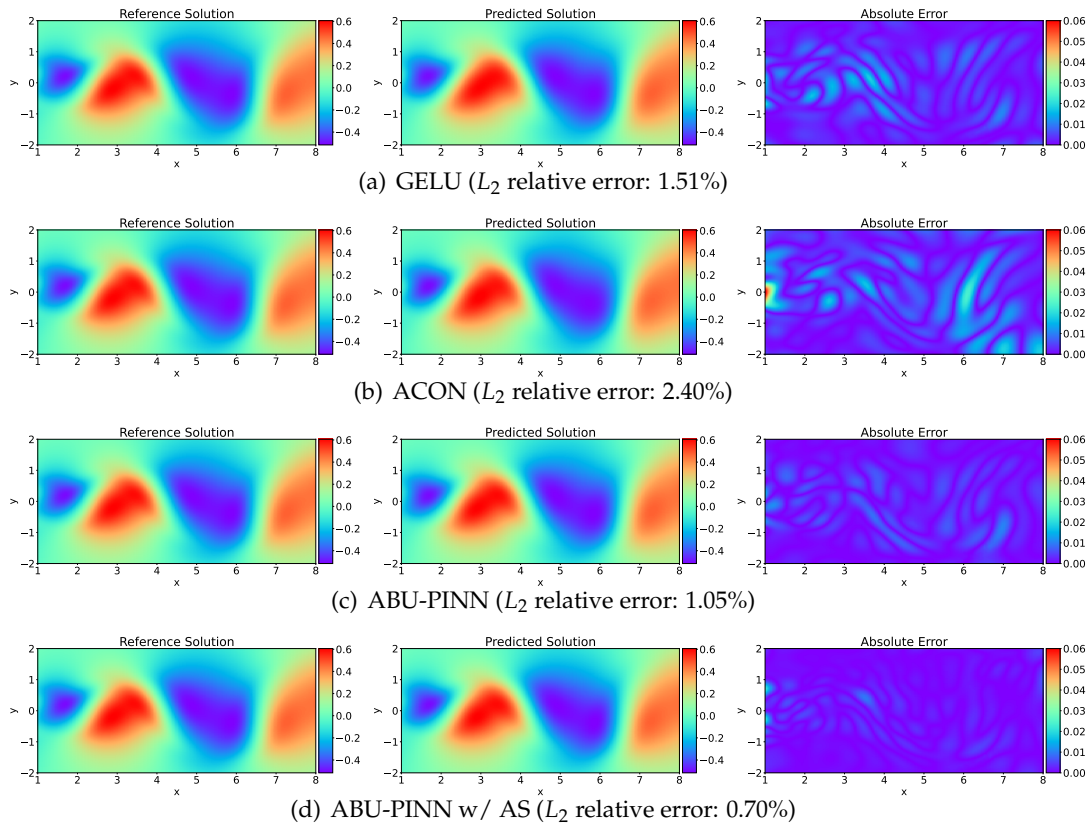
Furthermore, it is worth noting that the performance of ABU-PINN can be enhanced even further by incorporating the adaptive slope (AS) technique. We present the predictions of transverse velocity component v for the lid-driven cavity flow in Fig. 6 and the flow past a circular cylinder in Fig. 7. One can observe that the predicted solutions of ABU-PINN exhibit a favorable consistency with respect to the reference solutions. More visualization results can be found in Figs. 14 to 16 in the Appendix.

Ablation study

We ablate different design choices to provide a better understanding of the proposed method. For all ablation experiments, we deploy the layer-wise ABU-PINN to solve the convection equation, unless explicitly stated otherwise. The adaptive slope is deactivated for a clear comparison.

Candidate function set. Here we study the influence of candidate set by gradually increasing its size as shown in Table 9. We begin with the two most commonly-used activation functions, the sin and tanh functions. In this case, ABU-PINN is shown to achieve competitive result compared with its two candidate functions. Based on that, we observe that adding new activation functions could lead to a better performance. Surprisingly, the introduction of GELU can still improve the performance despite its poor results on this problem. We argue that this performance gain arises from the enlarged search space. We notice that the performance tends to saturate as the number of candidate functions increases. This indicates that ABU-PINN has a good robustness to the choice of candidate function set if this set contains activation functions with sufficient diversity. Notably, the additional parameters of ABU-PINN is negligible compared with total number of weights.

Figure 6: Flow in a lid-driven cavity, y-component of velocity v .

Figure 7: Flow past a circular cylinder, y-component of velocity v ($t=2.5$).Table 9: ABU-PINN ablation experiments on the convection equation. Comparisons of different candidate function set. We repeat each experiments 5 times and report the average L_2 relative error (%). Default settings are marked in gray.

Function set $\mathcal{F}$	#params	error (%)
{sin, tanh, GELU, Swish, Softplus}	16922	0.06$\pm$0.03
{sin, tanh, GELU, Swish}	16917	0.06$\pm$0.03
{sin, tanh, GELU}	16912	0.10 $\pm$ 0.12
{sin, tanh}	16907	0.33 $\pm$ 0.04
sin	16897	0.36 $\pm$ 0.15
tanh	16897	6.83 $\pm$ 4.79
GELU	16897	39.29 $\pm$ 35.51

Table 10: ABU-PINN ablation experiments on the convection equation. Comparisons of different search spaces. We repeat each experiments 5 times and report the average L_2 relative error (%). Default settings are marked in gray.

$G(\cdot)$	Init	Learnable	error (%)
identity	1/N	✓	0.22 ± 0.22
L_1 -norm	1/N	✓	1.18 ± 0.93
sigmoid	Zero	✓	0.89 ± 0.83
softmax	Zero	✓	0.06 ± 0.03
softmax	Zero	✗	1.60 ± 1.12
softmax	Random	✗	0.32 ± 0.23
softmax	Random	✓	0.09 ± 0.05

Gate functions. We also compare the softmax function with other gate functions in Table 10. The identity function $G(\alpha_i) = \alpha_i$ includes all linear combinations of candidate activation functions into the search space; while the sigmoid function $G(\alpha_i) = 1/(1 + \exp(-\alpha_i))$ restricts the coefficients to be between 0 and 1. We also consider the L_1 -normalization $G(\alpha_i) = \alpha_i / \sum_{j=1}^N |\alpha_j|$, which allows for negative coefficients and keeps the competition between candidate functions. The trainable $\{\alpha_i\}_{i=1}^N$ are initialized as zeros for sigmoid and softmax functions and as $1/N$ for identity and L_1 -normalization functions, where N denotes the number of candidate functions. One can observe that the results of softmax function are more stable and more accurate. This implies proper restrictions of the search space can work as a regularization to the learning of ABU-PINN.

Learnable coefficients of ABU-PINN. To demonstrate that the learnable coefficients of ABU-PINN is effective and necessary, we compare ABU-PINN to two variants with fixed coefficients. The first one initializes its parameters as zeros and the second one initializes its parameters from a standard normal distribution. As shown in Table 10, the automatically learned coefficients outperforms evenly distributed coefficients and random sampled coefficients by a large margin. One can also observe that the performance gap between the random and constant initialization is eliminated if these parameters are trainable, which demonstrates the effectiveness of ABU-PINN's optimization.

Network architectures. We validate the performance of our method across a range of network depths and widths. The results are summarized in Table 11. Remarkably, the proposed ABU-PINN consistently achieves the best performance across all investigated scenarios, surpassing standard activation functions and other adaptive activation functions. The empirical evidence, as depicted in Fig. 17, demonstrates that the training losses of ABU-PINN exhibit rapid and stable convergence across varying network depths and widths, affirming the robustness of our method in accommodating diverse network architectures.

Table 11: ABU-PINN ablation experiments on the Allen-Cahn equation. Comparisons of different network depths and widths. We repeat each experiments 5 times and report the average L_2 relative error (%). Default settings are marked in gray .

Method	Depth=4			Width=32		
	Width=32	Width=64	Width=96	Depth=4	Depth=6	Depth=8
sin	3.57±0.64	3.62±0.69	3.56±0.65	3.57±0.64	0.75±0.48	0.42±0.25
tanh	1.34±0.54	0.99±0.26	0.72±0.28	1.34±0.54	0.68±0.47	0.39±0.12
SLAF	33.93±9.97	32.56±12.51	28.04±5.14	33.93±9.97	8.55±1.66	3.30±0.92
PAU	43.83±15.18	33.61±13.86	26.93±16.40	43.83±15.18	21.25±19.14	32.00±21.90
ACON	3.88±1.82	3.08±1.16	2.31±0.44	3.88±1.82	3.34±2.42	2.46±2.74
ABU-PINN	0.76±0.26	0.66±0.04	0.45±0.07	0.76±0.26	0.26±0.01	0.22±0.01

Understanding ABU-PINN through the lens of neural tangent kernel

The neural tangent kernel is proposed to describe the evolution of neural networks under the assumptions of infinite network width and infinitesimal learning rate [62]. Its eigenvalues can be leveraged to analyse the rate of convergence as shown in previous works [11,62,63]. We show that the introduction of ABU-PINN makes the NTK learnable. Empirically, we observe the optimization of ABU-PINN adapts the NTK's eigenvalue spectrum to the underlying PDE system and leads to an improvement on the average eigenvalue, which partially explains the stable and fast convergence of ABU-PINN.

Brief introduction of NTK. The neural tangent kernel of a L -layer fully-connected network $u^{(L)}$ with parameters θ is defined as

$$\Theta^{(L)}(x, x') = \left\langle \frac{\partial u^{(L)}(x|\theta)}{\partial \theta}, \frac{\partial u^{(L)}(x'|\theta)}{\partial \theta} \right\rangle = \sum_{\theta_i} \frac{\partial u^{(L)}(x|\theta)}{\partial \theta_i} \frac{\partial u^{(L)}(x'|\theta)}{\partial \theta_i}, \quad (3.29)$$

where x and x' are two inputs. As demonstrated in [62], in the infinite-width limit and under suitable conditions, the NTK $\Theta^{(L)}$ converges to a deterministic kernel $\Theta_{\infty}^{(L)}$ at initialization and stays constant during the training. This limiting kernel can be represented as

$$\Theta_{\infty}^{(L)} = \sum_{l=1}^L \left(\Sigma^{(l)}(x, x') \prod_{l'=l+1}^L \dot{\Sigma}^{(l')}(x, x') \right), \quad (3.30)$$

$$\Sigma^{(l+1)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(l)})} [\sigma(f(x)) \sigma(f(x'))] + \beta^2, \quad (3.31)$$

$$\dot{\Sigma}^{(l+1)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(l)})} [\dot{\sigma}(f(x)) \dot{\sigma}(f(x'))] + \beta^2. \quad (3.32)$$

Note $\Sigma^{(1)}(x, x') = \frac{1}{n_0} x^T x' + \beta^2$. The expectations in Eqs. equation 3.31 and equation 3.32 are taken with respect to a centered Gaussian process f with covariance $\Sigma^{(l)}$; n_0 is the

dimension of input $\mathbf{x}$ and β is a scaling factor; σ and $\dot{\sigma}$ denote the activation function and its derivative, respectively. Note that the covariance $\Sigma^{(l)}$ depends on the choice of activation functions and so does the NTK $\Theta_{\infty}^{(L)}$.

ABU-PINN and the learnable NTK. For simplicity, we derive the NTK of standard neural networks with ABU-PINN. The NTK of PINNs with ABU-PINN can be derived in a similar way. Firstly, we assume that the optimization of neural networks with ABU-PINN can be decomposed into two phases, where we learn the coefficients of ABU-PINN in the first phase and then train the parameters of neural network in the second phase. This assumption is reasonable as the number of parameters of ABU-PINN is far less than those of networks and they quickly converge at the early stage of training. Empirically, we observe that a short warmup (2.5% of the whole schedule) is sufficient for ABU-PINN to learn suitable activation functions and to achieve competitive performance compared with the counterpart whose coefficients are updated during the whole schedule (0.58% vs. 0.60% on the Allen-Cahn equation). By decoupling the updates of ABU-PINN and networks, we find the second optimization phase is equivalent to the training of a standard network with a learned activation function, whose NTK is derived as

$$\bar{\Theta}_{\infty}^{(L)} = \sum_{l=1}^L \left(\left(\sum_{i=1}^N \sum_{j=1}^N G(\alpha_i) G(\alpha_j) \Sigma_{\sigma_i \sigma_j}^{(l)}(\mathbf{x}, \mathbf{x}') \right) \prod_{l'=l+1}^L \left(\sum_{i=1}^N \sum_{j=1}^N G(\alpha_i) G(\alpha_j) \dot{\Sigma}_{\sigma_i \sigma_j}^{(l')}(\mathbf{x}, \mathbf{x}') \right) \right), \quad (3.33)$$

$$\Sigma_{\sigma_i \sigma_j}^{(l+1)}(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(l)})} [\sigma_i(f(\mathbf{x})) \sigma_j(f(\mathbf{x}'))], \quad (3.34)$$

$$\dot{\Sigma}_{\sigma_i \sigma_j}^{(l+1)}(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(l)})} [\dot{\sigma}_i(f(\mathbf{x})) \dot{\sigma}_j(f(\mathbf{x}'))]. \quad (3.35)$$

We omit the factor β for convenience. One can observe that the covariance $\Sigma^{(l)}$ in Eq. (3.30) is replaced with a weighted sum of $\Sigma_{\sigma_i \sigma_j}^{(l)}$, which is calculated with different combinations of candidate activation functions. Note that the weights are learned in the first training phase. To conclude, the introduction of ABU-PINN leads to a learnable NTK which could be adapted to the underlying PDE.

The eigenvalue spectrum of NTK with ABU-PINN. Through the optimization of ABU-PINN, we can modify the limiting NTK $\bar{\Theta}_{\infty}^{(L)}$, which corresponds to a change in the convergence behavior of the neural network. As revealed by previous works [11, 64], the NTK's eigenvalues determine the convergence rate of the training loss. We then show empirically that the optimization of ABU-PINN has an effect on the NTK's eigenvalue spectrum. As we can see in Fig. 5(d), ABU-PINN leads to a greater average eigenvalue compared with the best standard activation function on the Allen-Cahn equation (5859 vs. 2407), which implies a faster convergence rate.

4 Summary and discussion

Despite the recent success, PINNs suffer from the ill-conditioned optimization. In this paper, we shed light on the relationship between activation functions and the optimization difficulty of PINNs. We reveal the high sensitivity of PINNs to the choice of activation functions and relate it to various characteristics of the underlying PDE system. To avoid the inefficient manual selection of activation functions, we propose to learn specialized activation functions automatically for PINNs to solve different problems. We compare different designs of adaptive activation functions and discuss their limitations in the context of PINNs. To remove obstacles to apply adaptive activation functions to PINNs, the proposed ABU-PINN tailors the idea of learning combinations of activation functions to the optimization of PDE-based constraints. Extensive experiments on a series of challenging benchmarks demonstrate the effectiveness of the proposed methods. In summary, our work presents a practical solution to the optimization difficulty from the perspective of activation functions. We hope our work could provide new insights and inspire further study on the convergence issue of PINNs from the point of view of network architecture design.

The limitations of our work are two-fold. The first limitation is that we only consider integer-order PDEs in the experiments, while different types of PDEs could have different effects on the training of adaptive activation functions. We leave the further exploration on fractional PDEs and stochastic PDEs for the future work. Another limitation is that we only focus on the empirical evaluation of adaptive activation functions, while theoretical analysis of the impact of PDE-based constraints on different algorithms is limited. We regard it as a promising direction to provide a rigorous theoretical analysis to gain insights into the activation function design for PINNs, and we will explore it in the future work.

Acknowledgments

The work is supported in part by the National Natural Science Foundation of China under Grants 62276150 and the Guoqiang Institute of Tsinghua University.

Appendices

A Standard activation functions

We compare several standard activation functions for solving a series of PDEs. The details are shown as follows.

- sin:

$$f(x) = \sin(\beta * x), \quad (\text{A.1})$$

where scaling factor β is set to 1 as default.

- cos:

$$f(x) = \cos(\beta * x), \quad (\text{A.2})$$

where β is set to 1.

- exp:

$$f(x) = e^{(\beta * x)} - 1, \quad (\text{A.3})$$

where β is set to 1 as default. For the **P1** setting of 1D Poisson's equation, β is set to 0.25 for better performance.

- tanh:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (\text{A.4})$$

- sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}. \quad (\text{A.5})$$

- Softplus [54,55]:

$$f(x) = \frac{1}{\beta} \log(1 + e^{\beta * x}), \quad (\text{A.6})$$

where $\beta = 1$. Softplus can be regarded as a smooth version of ReLU.

- the Exponential Linear Unit (ELU) [56]:

$$f(x) = \begin{cases} x, & \text{if } x \geq 0, \\ e^x - 1, & \text{if } x < 0. \end{cases} \quad (\text{A.7})$$

- the Gaussian Error Linear Unit (GELU) [57]:

$$f(x) = x * \Phi(x), \quad (\text{A.8})$$

where $\Phi(x)$ is the cumulative distribution function of the standard Gaussian distribution.

- Swish [34]:

$$f(x) = \frac{x}{1 + e^{-x}}. \quad (\text{A.9})$$

B Burgers' equation with varying viscosity

In this section, we conduct a comparative analysis of the performance obtained by different activation functions on Burgers' equation, considering varying viscosity values. As the viscosity approaches very small values, the smooth viscous solutions exhibit non-uniform convergence towards the appropriate discontinuous shock wave. This phenomenon presents a challenging problem for PINNs due to the unique complexities associated with capturing discontinuous features accurately [65].

Starting from the case in Section 3 with a viscosity of $0.01/\pi$, we investigate three additional cases with viscosity values of $\{0.007/\pi, 0.004/\pi, 0.001/\pi\}$. The same training configuration, as outlined in Section 3, is utilized for all cases. The number of training iteration with Adam optimization is set to 15000 for the case with a viscosity of $0.007/\pi$ and increased to 40000 for the other two cases. The results are summarized in Fig. 8 and Fig. 9. Notably, the proposed method consistently outperforms standard activation functions as the viscosity value is decreased, showcasing its superior efficacy. As a prominent example, the proposed ABU-PINN successfully achieves accurate predictions for the case with a viscosity of $0.004/\pi$. In contrast, the sin activation function fails to produce satisfactory results, while the tanh activation function yields distorted predictions around the time of 0.5. However, when the viscosity is further decreased to $0.001/\pi$, neither standard activation functions nor ABU-PINN can obtain satisfactory predictions. A plausible explanation for this observation is that the continuous nature of the proposed method may restrict its capability to enhance the current framework of PINNs in effectively capturing discontinuities.

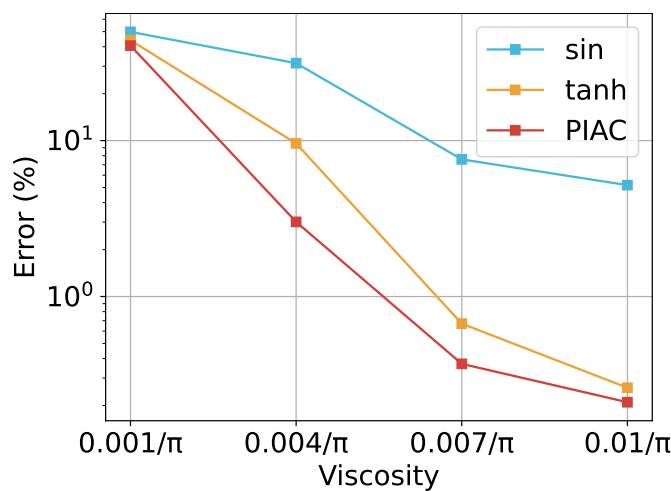


Figure 8: The performance on Burgers' equation with varying viscosity. We repeat each experiments 3 times and report the average L_2 relative error (%)

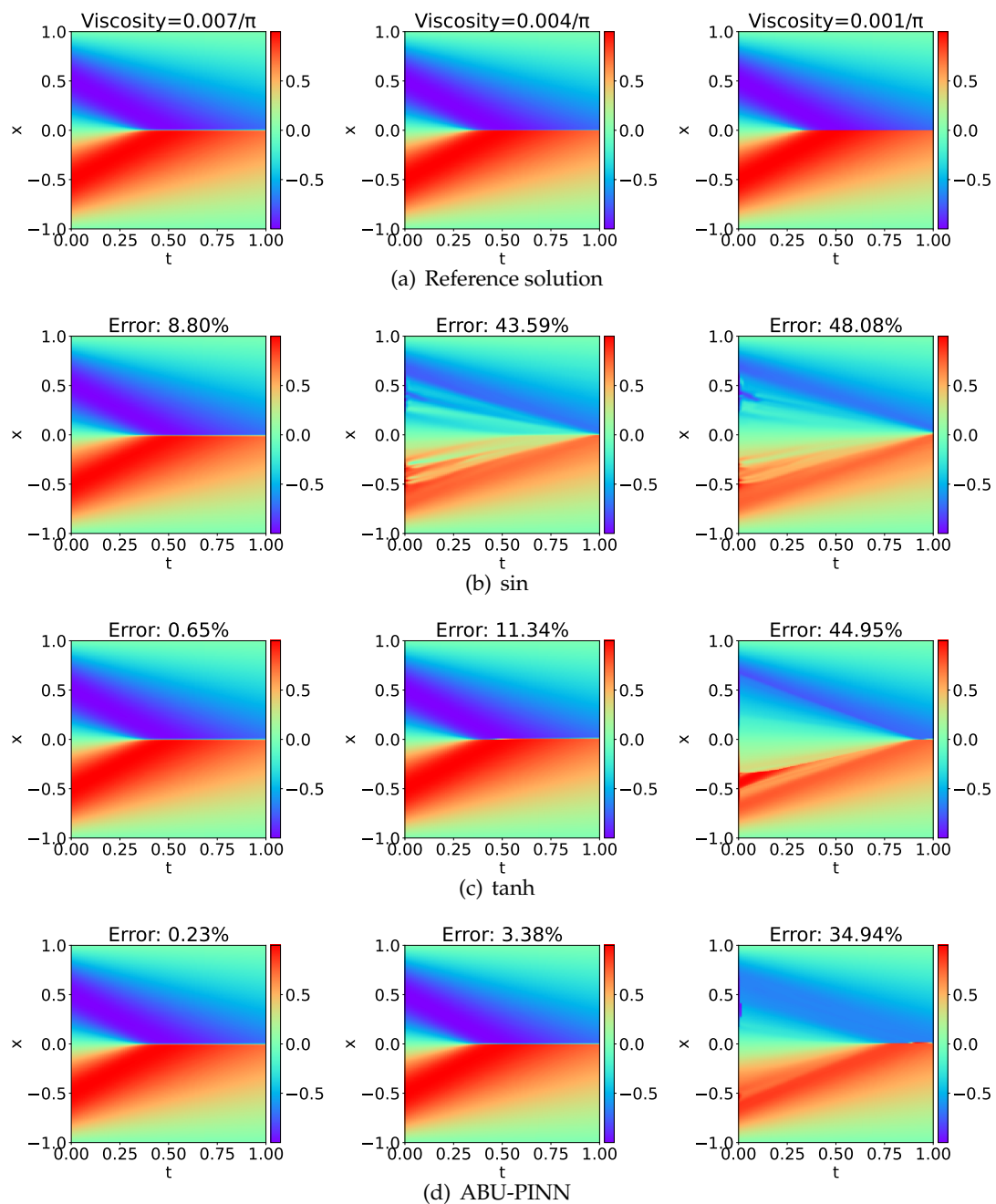


Figure 9: Predicted solutions for Burgers' equation with viscosity values of $0.007/\pi$ (left), $0.004/\pi$ (middle) and $0.001/\pi$ (right).

C Visualization results

We present the predicted solutions of ABU-PINN for Burgers' equation in Fig. 10, KdV equation in Fig. 11, and Cahn-Hilliard equation in Fig. 12. We show the training losses of

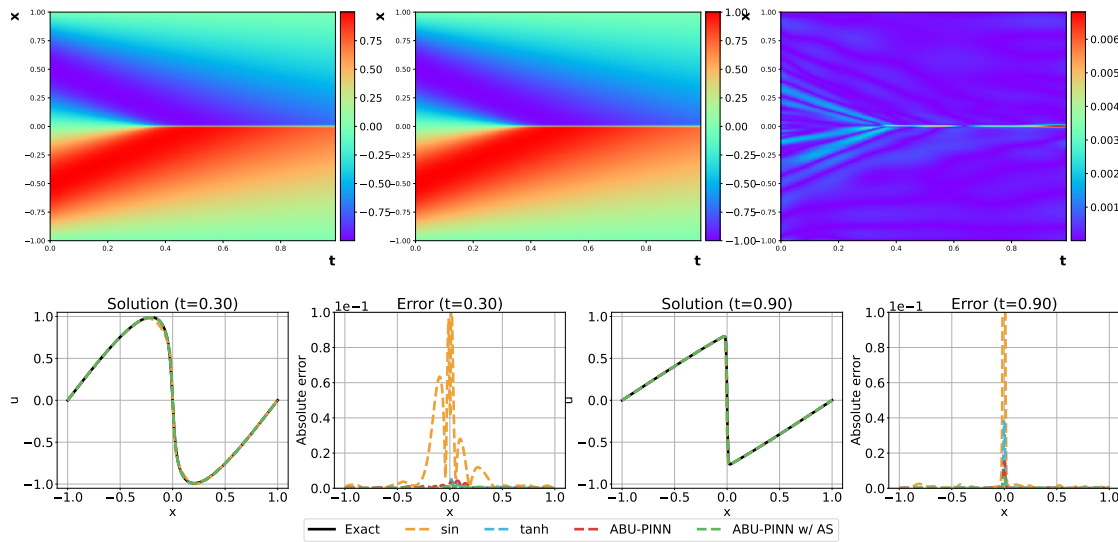


Figure 10: The burgers' equation. Top: the exact solution (left), the predictions of ABU-PINN (middle), and the absolute error between them (right). Bottom: predicted solutions at different time snapshots.

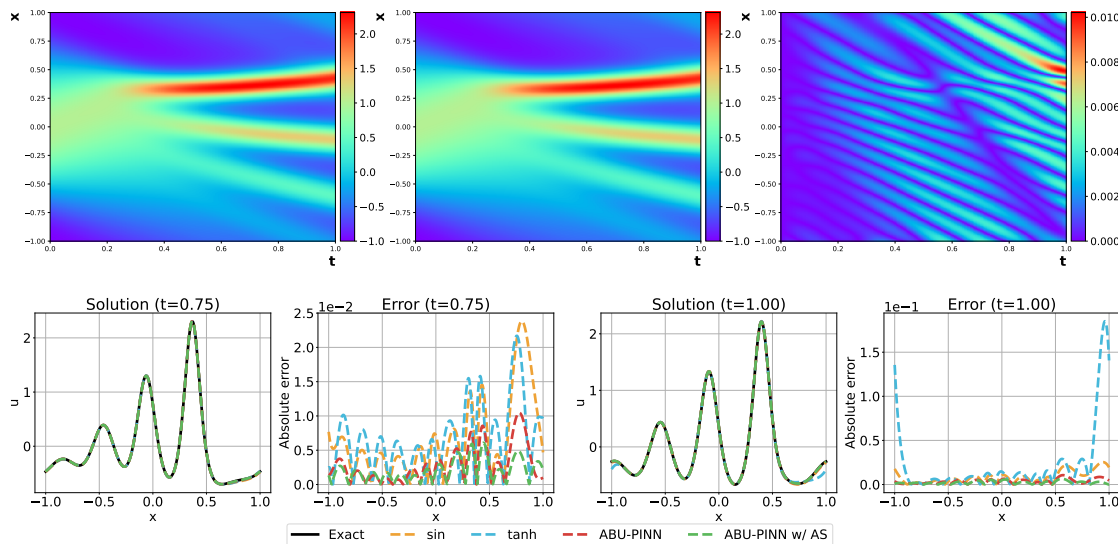


Figure 11: The KdV equation. Top: the exact solution (left), the predictions of ABU-PINN (middle), and the absolute error between them (right). Bottom: predicted solutions at different time snapshots.

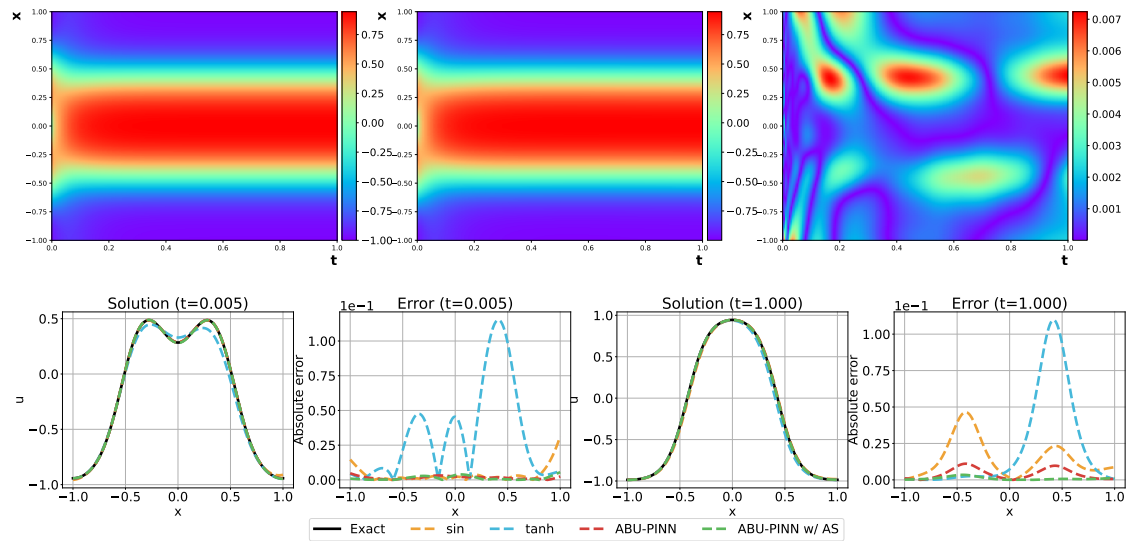


Figure 12: The Cahn-Hilliard equation. Top: the exact solution (left), the predictions of ABU-PINN (middle), and the absolute error between them (right). Bottom: predicted solutions at different time snapshots.

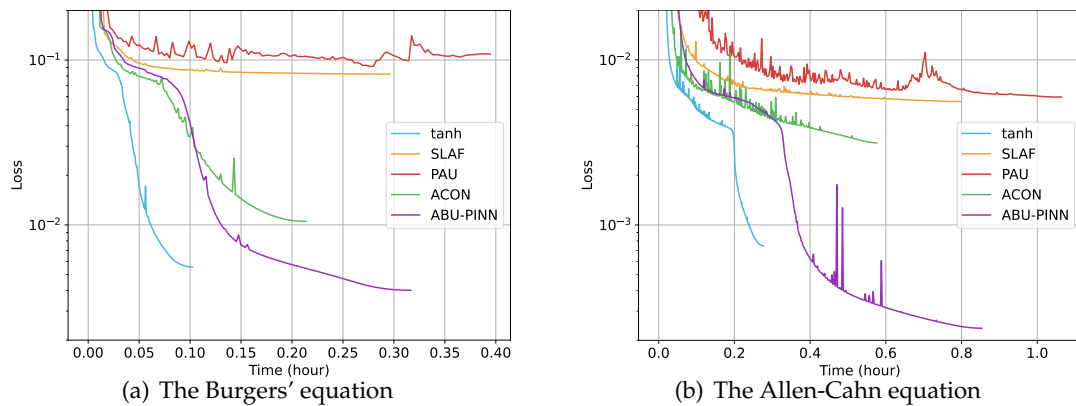
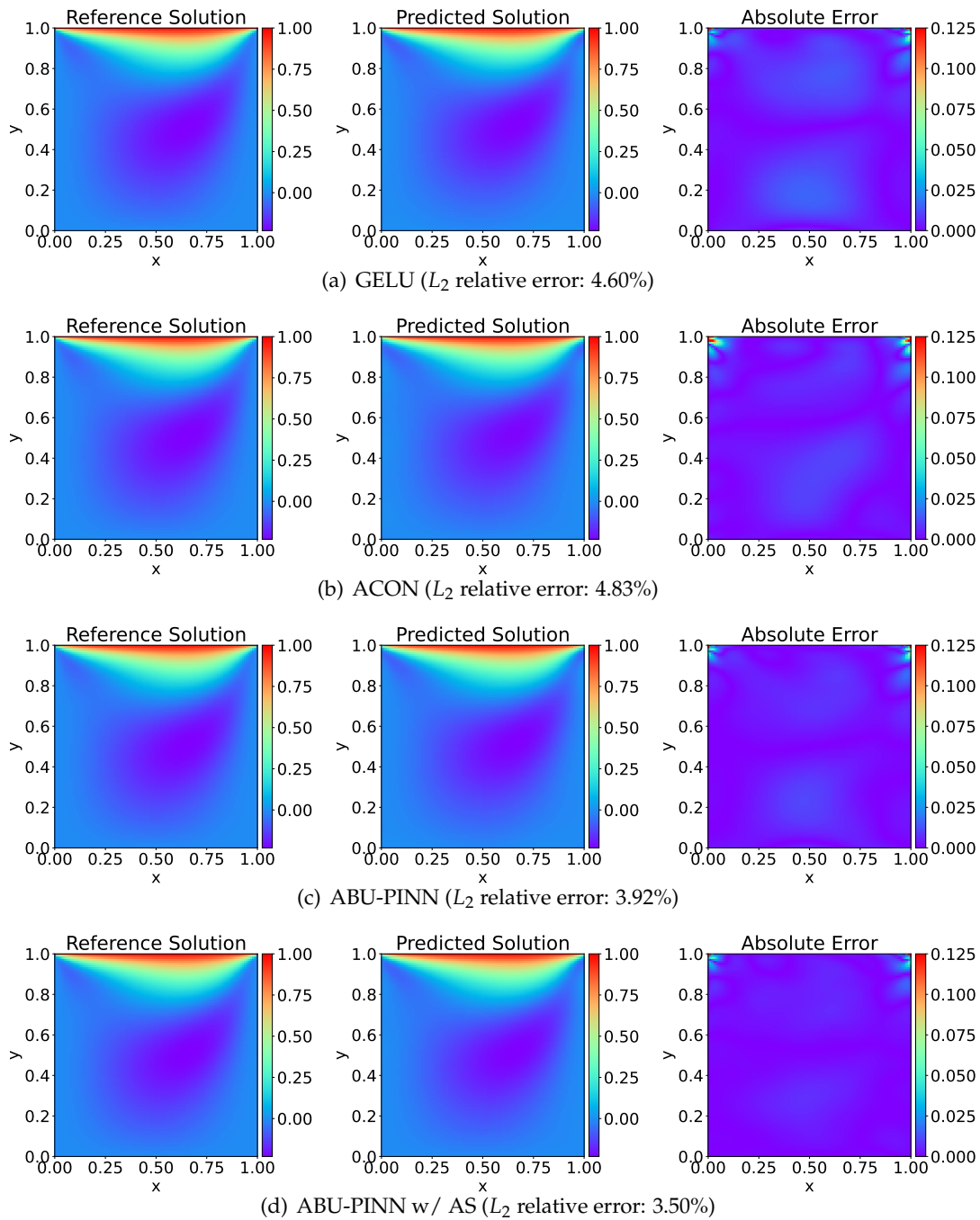
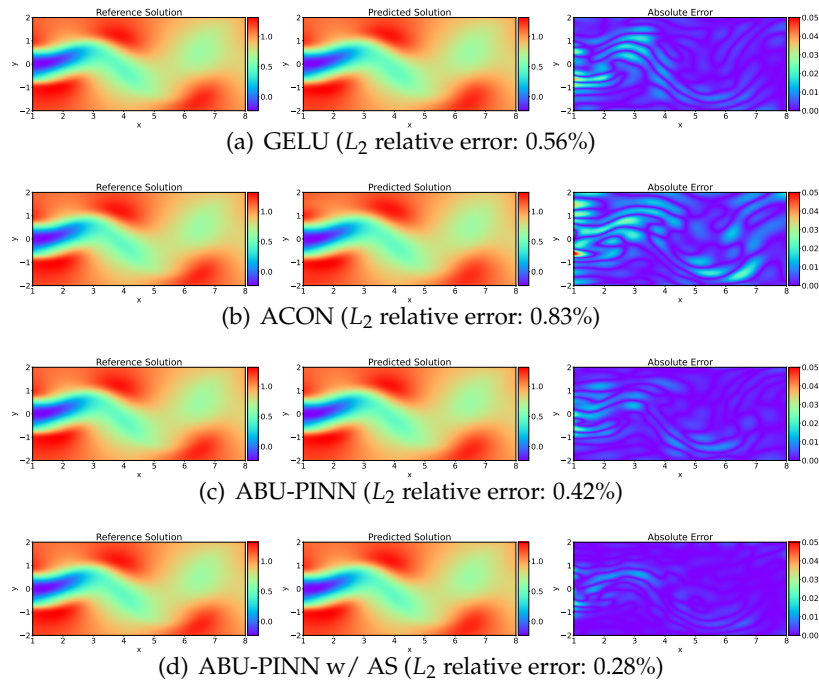
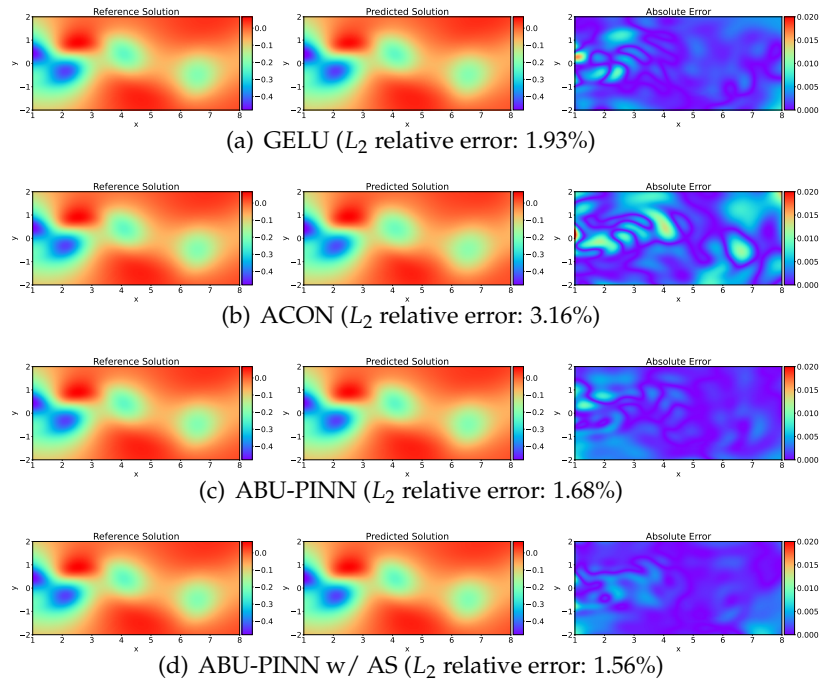


Figure 13: The training losses of different activation functions against training time (hour).

different activation functions against training time in Fig. 13. Fig. 14 shows the predicted x -component of velocity u in the case of flow in a lid-driven cavity. For the flow past a circular cylinder, we presents the reference and predicted solutions of stream-wise velocity component u and the pressure filed p in Fig. 15 and Fig. 16, respectively. Additionally, we provide the training curves of different activation functions across various network depths and width in Fig. 17.

Figure 14: Flow in a lid-driven cavity, x -component of velocity u .

Figure 15: Flow past a circular cylinder, x -component of velocity u ($t=2.5$).Figure 16: Flow past a circular cylinder, pressure p ($t=2.5$).

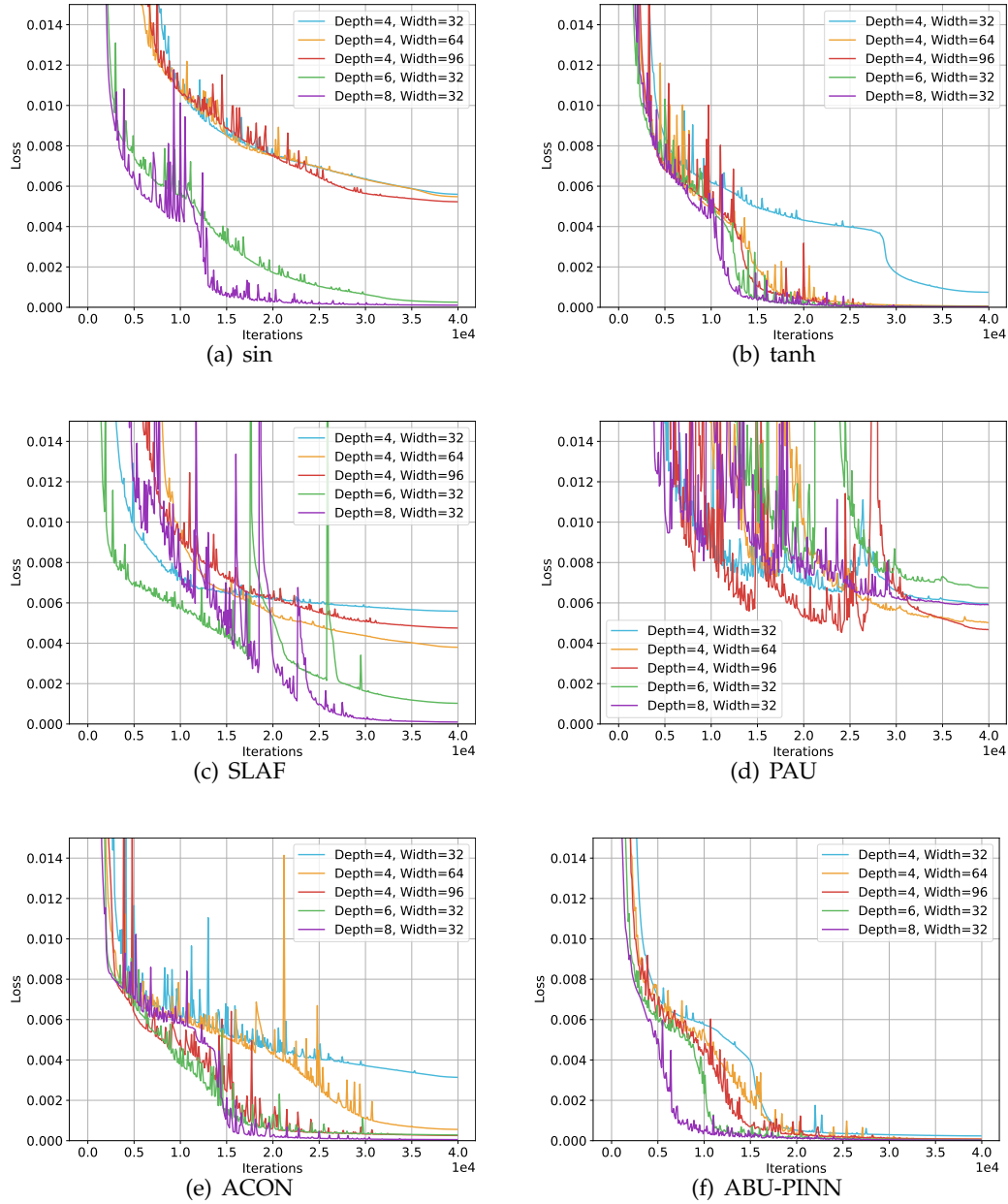


Figure 17: The training losses of different activation functions across various network depths and widths.

References

- [1] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational physics* 378 (2019) 686–707.
- [2] Y. Chen, L. Lu, G. E. Karniadakis, L. Dal Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, *Optics Express* 28 (8) (2020) 11618–11633.
- [3] Y. Shin, J. Darbon, G. E. Karniadakis, On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs, *Communications in Computational Physics* 28 (5) (2020) 2042–2074.
- [4] L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, S. G. Johnson, Physics-informed neural networks with hard constraints for inverse design, *SIAM Journal on Scientific Computing* 43 (6) (2021) B1105–B1132.
- [5] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, *Nature Reviews Physics* 3 (6) (2021) 422–440.
- [6] Y. Jiao, Y. Lai, D. Li, X. Lu, F. Wang, Y. Wang, J. Z. Yang, A rate of convergence of physics informed neural networks for the linear second order elliptic PDEs, *Communications in Computational Physics* 31 (4) (2022) 1272.
- [7] E. Weinan, B. Yu, The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems, *Communications in Mathematics and Statistics* 1 (6) (2018) 1–12.
- [8] L. Zhang, T. Luo, Y. Zhang, W. E, Z.-Q. John Xu, Z. Ma, MOD-Net: A machine learning approach via model-operator-data network for solving PDEs, *Communications in Computational Physics* 32 (2) (2022) 299–335.
- [9] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM Journal on Scientific Computing* 43 (5) (2021) A3055–A3081.
- [10] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Advances in Neural Information Processing Systems* 34 (2021).
- [11] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: A neural tangent kernel perspective, *Journal of Computational Physics* 449 (2022) 110768.
- [12] A. F. Psaros, K. Kawaguchi, G. E. Karniadakis, Meta-learning PINN loss functions, *Journal of Computational Physics* 458 (2022) 111121.
- [13] J. Yu, L. Lu, X. Meng, G. E. Karniadakis, Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems, *Computer Methods in Applied Mechanics and Engineering* 393 (2022) 114823.
- [14] C. Wang, S. Li, D. He, L. Wang, Is l^2 physics-informed loss always suitable for training physics-informed neural network?, *arXiv preprint arXiv:2206.02016* (2022).
- [15] C. L. Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, *Communications in Computational Physics* 29 (3) (2020).
- [16] L. McClenny, U. Braga-Neto, Self-adaptive physics-informed neural networks using a soft attention mechanism, *arXiv preprint arXiv:2009.04544* (2020).
- [17] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis, DeepXDE: A deep learning library for solving differential equations, *SIAM Review* 63 (1) (2021) 208–228.
- [18] C. Wu, M. Zhu, Q. Tan, Y. Kartha, L. Lu, A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks, *arXiv preprint*

arXiv:2207.10289 (2022).

- [19] A. D. Jagtap, E. Kharazmi, G. E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, *Computer Methods in Applied Mechanics and Engineering* 365 (2020) 113028.
- [20] A. D. Jagtap, G. E. Karniadakis, Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, *Communications in Computational Physics* 28 (5) (2020) 2002–2041.
- [21] B. Moseley, A. Markham, T. Nissen-Meyer, Finite basis physics-informed neural networks (FBPINNs): A scalable domain decomposition approach for solving differential equations, *arXiv preprint arXiv:2107.07871* (2021).
- [22] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [23] G. Huang, Z. Liu, L. Van Der Maaten, K. Q. Weinberger, Densely connected convolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4700–4708.
- [24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, *Advances in Neural Information Processing Systems* 30 (2017).
- [25] M. Raissi, Z. Wang, M. S. Triantafyllou, G. E. Karniadakis, Deep learning of vortex-induced vibrations, *Journal of Fluid Mechanics* 861 (2019) 119–137.
- [26] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, G. Wetzstein, Implicit neural representations with periodic activation functions, *Advances in Neural Information Processing Systems* 33 (2020) 7462–7473.
- [27] Z.-Q. J. Xu, Frequency principle: Fourier analysis sheds light on deep neural networks, *Communications in Computational Physics* 28 (5) (2020) 1746–1767.
- [28] X.-A. Li, Z.-Q. John Xu, L. Zhang, A multi-scale DNN algorithm for nonlinear elliptic equations with multiple scales, *Communications in Computational Physics* 28 (5) (2020) 1886–1906.
- [29] J. C. Wong, C. Ooi, A. Gupta, Y.-S. Ong, Learning in sinusoidal spaces with physics-informed neural networks, *IEEE Transactions on Artificial Intelligence* (2022).
- [30] R. H. Hahnloser, R. Sarpeshkar, M. A. Mahowald, R. J. Douglas, H. S. Seung, Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit, *nature* 405 (6789) (2000) 947–951.
- [31] K. Jarrett, K. Kavukcuoglu, M. Ranzato, Y. LeCun, What is the best multi-stage architecture for object recognition?, in: *2009 IEEE 12th International Conference on Computer Vision*, IEEE, 2009, pp. 2146–2153.
- [32] V. Nair, G. E. Hinton, Rectified linear units improve restricted Boltzmann machines, in: *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 2010, pp. 807–814.
- [33] L. Lu, Y. Shin, Y. Su, G. E. Karniadakis, Dying ReLU and initialization: Theory and numerical examples, *Communications in Computational Physics* 28 (5) (2020) 1671–1706.
- [34] P. Ramachandran, B. Zoph, Q. V. Le, Searching for activation functions, *arXiv preprint arXiv:1710.05941* (2017).
- [35] F. Agostinelli, M. Hoffman, P. Sadowski, P. Baldi, Learning activation functions to improve deep neural networks, *arXiv preprint arXiv:1412.6830* (2014).
- [36] J. Li, T. Zhang, W. Luo, J. Yang, X.-T. Yuan, J. Zhang, Sparseness analysis in the pretraining of

- deep neural networks, *IEEE Transactions on Neural Networks and Learning Systems* 28 (6) (2016) 1425–1438.
- [37] Y. Zhou, Z. Zhu, Z. Zhong, Learning specialized activation functions with the piecewise linear unit, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 12095–12104.
 - [38] M. Goyal, R. Goyal, B. Lall, Learning activation functions: A new paradigm of understanding neural networks. *arxiv* 2019, *arXiv preprint arXiv:1906.09529* (2019).
 - [39] A. Molina, P. Schramowski, K. Kersting, Padé activation units: End-to-end learning of flexible activation functions in deep networks, *arXiv preprint arXiv:1907.06732* (2019).
 - [40] N. Ma, X. Zhang, M. Liu, J. Sun, Activate or not: Learning customized activation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 8032–8042.
 - [41] I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio, Maxout networks, in: *International Conference on Machine Learning*, PMLR, 2013, pp. 1319–1327.
 - [42] M. Dushkoff, R. Ptucha, Adaptive activation functions for deep networks, *Electronic Imaging* 2016 (19) (2016) 1–5.
 - [43] S. Qian, H. Liu, C. Liu, S. Wu, H. San Wong, Adaptive activation functions in convolutional neural networks, *Neurocomputing* 272 (2018) 204–212.
 - [44] F. Manessi, A. Rozza, Learning combinations of activation functions, in: *2018 24th International Conference on Pattern Recognition (ICPR)*, IEEE, 2018, pp. 61–66.
 - [45] L. R. Süttfeld, F. Brieger, H. Finger, S. Füllhase, G. Pipa, Adaptive blending units: Trainable activation functions for deep neural networks, in: *Science and Information Conference*, Springer, 2020, pp. 37–50.
 - [46] A. D. Jagtap, Y. Shin, K. Kawaguchi, G. E. Karniadakis, Deep Kronecker neural networks: A general framework for neural networks with adaptive activation functions, *Neurocomputing* 468 (2022) 165–180.
 - [47] A. D. Jagtap, K. Kawaguchi, G. E. Karniadakis, Adaptive activation functions accelerate convergence in deep and physics-informed neural networks, *Journal of Computational Physics* 404 (2020) 109136.
 - [48] A. D. Jagtap, K. Kawaguchi, G. E. Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks, *Proceedings of the Royal Society A* 476 (2239) (2020) 20200334.
 - [49] G. Pang, L. Lu, G. E. Karniadakis, fPINNs: Fractional physics-informed neural networks, *SIAM Journal on Scientific Computing* 41 (4) (2019) A2603–A2626.
 - [50] D. Zhang, L. Lu, L. Guo, G. E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, *Journal of Computational Physics* 397 (2019) 108850.
 - [51] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, Automatic differentiation in machine learning: A survey, *Journal of Machine Learning Research* 18 (2018) 1–43.
 - [52] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, JMLR Workshop and Conference Proceedings, 2010, pp. 249–256.
 - [53] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, *arXiv preprint arXiv:1412.6980* (2014).
 - [54] C. Dugas, Y. Bengio, F. Bélisle, C. Nadeau, R. Garcia, Incorporating second-order functional knowledge for better option pricing, *Advances in Neural Information Processing Systems* 13 (2000).

- [55] X. Glorot, A. Bordes, Y. Bengio, Deep sparse rectifier neural networks, in: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, JMLR Workshop and Conference Proceedings, 2011, pp. 315–323.
- [56] D.-A. Clevert, T. Unterthiner, S. Hochreiter, Fast and accurate deep network learning by exponential linear units (ELUs), arXiv preprint arXiv:1511.07289 (2015).
- [57] D. Hendrycks, K. Gimpel, Bridging nonlinearities and stochastic regularizers with Gaussian error linear units, CoRR, abs/1606.08415 3 (2016).
- [58] S. Elfving, E. Uchibe, K. Doya, Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, Neural Networks 107 (2018) 3–11.
- [59] N. Zobeiry, K. D. Humfeld, A physics-informed machine learning approach for solving heat transfer equation in advanced manufacturing and engineering applications, Engineering Applications of Artificial Intelligence 101 (2021) 104232.
- [60] R. Matthey, S. Ghosh, A novel sequential method to train physics informed neural networks for Allen Cahn and Cahn Hilliard equations, Computer Methods in Applied Mechanics and Engineering 390 (2022) 114474.
- [61] R. H. Byrd, P. Lu, J. Nocedal, C. Zhu, A limited memory algorithm for bound constrained optimization, SIAM Journal on Scientific Computing 16 (5) (1995) 1190–1208.
- [62] A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: Convergence and generalization in neural networks, Advances in Neural Information Processing Systems 31 (2018).
- [63] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, Advances in Neural Information Processing Systems 33 (2020) 7537–7547.
- [64] S. Wang, S. Sankaran, P. Perdikaris, Respecting causality is all you need for training physics-informed neural networks, arXiv preprint arXiv:2203.07404 (2022).
- [65] O. Fuks, H. A. Tchelepi, Limitations of physics informed machine learning for nonlinear two-phase transport in porous media, Journal of Machine Learning for Modeling and Computing 1 (1) (2020).